Contents

Morpheus: On-Device Predictive Autocompletion for Basque Using State Space Models

Preprint — July 2026

Authors: Xabier Ezpeleta

Code & Models: `github.com/itzune/morpheus-mamba`

Live Demo: Docker-based local deployment supporting both Smart Compose–style ghost text and smartphone keyboard–style word chips

Keywords: predictive autocompletion, Basque, Euskara, Mamba, State Space Models, agglutinative languages, on-device inference, keystroke savings, evaluation methodology, next-word prediction, subword tokenization

---

Abstract

We present **Morpheus**, an on-device predictive autocompletion system for Basque (Euskara), a low-resource agglutinative language with rich morphological inflection. The research question is whether a Gmail Smart Compose–equivalent multi-token continuation system — which Google serves from Cloud TPUs using an ~80M-parameter LSTM — can run **entirely on a consumer laptop**, with zero network calls, for use in text editors and desktop applications. The system supports two prediction paradigms: **multi-token continuation** (Smart Compose–style inline ghost text, accepted with Tab) for the desktop use case, and **next-word prediction** (smartphone keyboard–style discrete word chips, accepted with a tap) for the mobile use case, where Gboard's 1.4M/1.4MB on-device model defines the target footprint. Traditional approaches—LSTMs and distilled Transformers—each present tradeoffs between inference latency, model quality, and engineering complexity for agglutinative languages. We evaluate three candidate architectures (xLSTM, distilled Transformer, Mamba-2 SSM) against a set of constraints: P90 latency <= 50ms on consumer CPU, zero network calls, and <= 300 MB on-disk. We select Mamba-2, a Selective State Space Model offering constant-memory O(1) per-step inference with no KV cache overhead — the same property

(no KV cache) that led Google to choose LSTM over Transformer for Smart Compose, but achieved here with a modern architecture that runs on-device rather than requiring data-center TPUs.

Our Morpheus-Small model (91M parameters, Q4_K_M quantization, 55 MB) is trained on 4.62B tokens of curated Basque text (11 sources, 128M lines) using a 4K-vocabulary SentencePiece Unigram tokenizer. **A controlled vocabulary-size ablation experiment** across 4K, 8K, 16K, and 32K reveals a categorical divide: at 4K, Basque verbal agreement morphemes (`zki`, `zu`, `te`) emerge as independent, reusable tokens (e.g., `dizkizut -> di zki zu t`); at 32K, entire polysynthetic verbs are stored as opaque atomic units (e.g., `|dizkizut` as a single token). The widely-used fertility metric favors the 32K tokenizer (1.50 vs 4K's 2.05), but this is achieved exactly by fusing morphemes — the mechanism that destroys morphological generalization. We argue that **fertility is a confound, not a quality metric, for agglutinative tokenizer evaluation**, and confirm the QuechuaTok finding that vocabulary size, not algorithm choice, is the primary driver.

We introduce a multi-metric evaluation suite — Character Savings Rate (CSR) with bootstrap confidence intervals, Morpheme Boundary Accuracy (MorphAcc), Case Paradigm Completion, Perplexity (PPL), blinded human A/B evaluation, and completion logging with replay — designed specifically for agglutinative autocomplete and tagged to the prediction paradigm each measures. After full training (76K steps, ~10B tokens, 14.9 hours), the best checkpoint (step 74K) achieves **25.3% CSR** (95% CI [24.0%, 26.5%]) on 300 held-out sentences, **76% MorphAcc**, and **held-out PPL of 7.13**. A key methodological finding emerges: **PPL is the only metric that consistently produces coherent, reliable signal**, unambiguously confirming the later checkpoint is better (7.56 -> 7.17 -> 7.13, all 14 evaluation files agree). The autocomplete-specific metrics proved difficult to implement correctly and yielded weak data: CSR requires token-ID prompts to avoid BOS/tokenizer divergence bugs (string prompts gave ~4% vs ~28% with token IDs), and even correctly implemented produces overlapping confidence intervals that cannot distinguish competent models; the blinded human A/B (p=0.82) is underpowered at n=30. The next-word replay does favor 54K (Top-1 60->80%) but conflates model quality with inference engineering. **In practice, PPL is the metric we trust for checkpoint comparison; the autocomplete metrics serve as sanity checks rather than ranking tools at this scale.** We show that CSR is a **lower bound** for agglutinative autocomplete: exact-match gold cannot credit valid alternative continuations, causing the metric to saturate once models reach competence — confirmed by the fact that CSR barely moves from step 54K to the converged step 74K (25.23% -> 25.26%) despite continued PPL improvement. Furthermore, a sentence-level typing simulation reveals a **CSR paradox**: the model's native Basque achieves the lowest simulated CSR (19.2%), below English (26.3%) and Spanish (30.9%) which represent <1% of training data — a structural artifact of agglutinative word length, not a model deficiency, confirming that CSR penalizes the very language such systems are designed to serve. Furthermore, we demonstrate that **unblinded qualitative assessment is subject to expectation bias** — the expert's unblinded impression that the later checkpoint was "much better" was not borne out by blinded evaluation. We further document five inference engineering strategies — retokenization fallback, sticky merge (candidate carry-forward), top-k exceeding display-k, next-word candidate extraction, and completion logging with replay — that address failure modes unique to deploying subword-tokenized models as interactive next-word keyboards in agglutinative languages. A cross-model baseline comparison using **Bits Per Character (BPC)** — a tokenizer-independent metric — shows that Morpheus (91M, BPC 0.970) outperforms GPT-2 eus-euscrawl (124M, BPC 0.981) despite having fewer parameters, and approaches the performance of Latxa-Qwen3.5-2B (1.88B, BPC 0.822) at 1/20th the parameter count and 1/22nd the deployment size (55 MB vs ~1.2 GB). A data scaling analysis situates our 110:1 tokens-to-parameters ratio within the scaling law literature (Chinchilla 20:1, Mosaic 190:1, MiniCPM 192:1), finding that the model converged

at ~8.8B tokens due to data quality constraints rather than data quantity — suggesting that for low-resource languages, aggressive quality filtering of a 2–5B token corpus would be more efficient than maximizing raw token count. These findings have implications for evaluation methodology in agglutinative language modeling.

---

1. Introduction

Predictive autocompletion—suggesting the next words as a user types—is a mature technology for high-resource languages. Google's Gmail Smart Compose (Chen et al., 2019) serves billions of suggestions daily from an ~80M-parameter LSTM running on Cloud TPUs in data centers. Gboard (Hard et al., 2018) provides on-device next-word prediction on smartphones with a 1.4M-parameter, 1.4 MB model. Apple's QuickType has been integrated into iOS since 2014. However, these systems target languages with simple morphology (English, Spanish, Chinese) where next-word prediction is primarily a collocation problem — and none of them run a Smart Compose–equivalent multi-token continuation system entirely on-device.

The research question. Smart Compose demonstrates that multi-token ghost-text autocompletion is valuable and deployable at scale — but only from the cloud. Can an equivalent system run **entirely locally on a consumer laptop**, without any network calls, for use in text editors and desktop applications? This is the primary question Morpheus investigates. A secondary question — whether such a system can also power **mobile next-word prediction** (the Gboard paradigm) — requires a much smaller model than Smart Compose's 80M or our 91M; Gboard's 1.4M/1.4MB model defines the target footprint for mobile, and our current model serves as a starting point for that trajectory rather than a finished mobile solution.

Basque (Euskara) is fundamentally different. As a language isolate with agglutinative morphology, a single Basque verb can encode subject, direct object, indirect object, tense, mood, and aspect through suffix chains (e.g., *ikusiko zenizkidakeen* — "you would have been able to see them to me"). A Basque noun takes 12+ case suffixes, plus number and definiteness marking (e.g., *etxeetaraino* — "up to the houses"). This means that predicting the next word is not just about collocations—it requires **morphological productivity**: the ability to generate grammatically correct suffix sequences that the model has never seen as a unit during training.

Recent work on agglutinative language modeling confirms this challenge. QuechuaTok (Contreras, 2026) showed that standard BPE tokenizers achieve only 6.67% morpheme boundary accuracy on Quechua, while morphology-aware tokenization reaches 83.33%. Lane et al. (2022) demonstrated that morph-based word completion for Plains Cree requires explicit morphological segmentation to be usable.

Morpheus is designed for this challenge. We build an on-device predictive autocompletion system for Basque that supports two complementary prediction paradigms — **multi-token continuation** (Smart Compose–style inline ghost text for desktop/text-editor use, accepted with a single Tab) and **next-word prediction** (smartphone keyboard–style discrete word chips, accepted with a tap) — and that:

1. Runs **entirely locally** on consumer hardware (no cloud dependency, privacy-first)
2. Provides suggestions within **<= 50ms P90 latency** on a standard x86-64 CPU
3. Achieves **meaningful keystroke savings** for practical utility

4. Handles **Basque morphology** — not just memorized collocations but productive case suffix prediction
5. Supports **Euskañol** (Basque-Spanish code-switching), common in informal communication
6. Fits within **<= 300 MB** on disk (feasible for browser extensions and desktop applications)
7. Supports **both prediction paradigms** — inline ghost text for desktop/text-editor writing and discrete word chips for mobile keyboard-style input — with inference engineering tailored to each

The 91M model is sized for the **desktop multi-token continuation** use case: comparable to Smart Compose's 80M, but running on-device rather than on Cloud TPUs. The **mobile next-word prediction** use case requires a substantially smaller model to match Gboard's 1.4 MB footprint; we document the inference engineering strategies (Section 5.5) that would apply equally to such a distilled mobile variant.

This paper documents our architecture selection process, training pipeline, evaluation methodology, and final results after complete training (76K steps, best checkpoint step 74K). A key finding is a **CSR paradox**: the model's native Basque achieves the lowest simulated keystroke savings, below non-target languages representing <1% of training data — a structural artifact of agglutinative word length that demonstrates CSR is a biased metric for the very languages such systems are built to serve.

---

2. Related Work

2.1 Predictive Autocompletion

Two prediction paradigms dominate real-world autocomplete systems, and they differ not only in user experience but also in deployment constraints, model scale, and the engineering challenges they impose:

Multi-token continuation, exemplified by Google's Gmail Smart Compose (Chen et al., 2019), generates a multi-word suggestion displayed inline as ghost text and accepted with a single keystroke (Tab). Smart Compose is a **server-side** system: its production model is an LSTM-2-1024 at approximately **80M parameters**, trained on approximately **8 billion English email messages** (80/20 train/test split) using a word-level vocabulary of 50K English words, served on Cloud TPUs to over 1.5 billion users with a P90 latency under 60ms (including network round-trip). The paper reports that despite Transformers achieving better perplexity, the LSTM was chosen for production because Transformer self-attention requires maintaining keys and values from all previous decoding steps, making per-step latency grow with context length — incompatible with the strict real-time constraint. This is precisely the KV-cache problem that motivates our use of Mamba-2 (Section 3.3): Google solved it with data-center TPUs; we solve it with a fundamentally linear-time architecture that runs **on-device**. The data scale difference is instructive: Google had access to 8 billion real emails from 1.5 billion users — a data volume available only for a high-resource language like English at Google's scale. For Basque, the total extant high-quality text is finite and orders of magnitude smaller, which directly shapes our data scaling analysis (Section 6.16). Moreover, Smart Compose's training data is **domain-matched** — trained on emails, deployed for writing emails — which means the model's learned distribution directly matches the user's writing context. This domain match is itself a UX advantage that Morpheus lacks: we train on general Basque prose (Wikipedia, news, literature) but deploy as a general-purpose text editor, producing corpus-induced

artifacts (Section 6.12) where the model over-predicts encyclopedic patterns (dates, numbers) rather than conversational continuations.

Next-word prediction, exemplified by Gboard (Hard et al., 2018) and Apple QuickType, offers discrete word chips above the keyboard, each accepted with a tap. Gboard's on-device next-word prediction model is an LSTM variant with only **1.4M parameters**, quantized to **1.4 MB** — small enough to run on any smartphone with zero network calls. This extreme compression is necessary because mobile keyboards operate under far tighter resource constraints than desktop text editors: limited RAM, battery sensitivity, and the need to coexist with other running applications. Gboard uses a word-level vocabulary (50K English words) and federated learning to personalize on-device; it does not handle subword morphology.

The scale difference between these two production systems — 80M server-side (Smart Compose) vs. 1.4M on-device (Gboard) — reflects the deployment context: desktop/email multi-token continuation can afford a larger model served from the cloud, while mobile next-word prediction demands an ultra-lightweight model running locally. **GitHub Copilot** (Fu & Mogensen, 2026; Cheney, 2025) applies the Smart Compose paradigm to code: multi-token ghost-text completions in the IDE, accepted with Tab. Like Smart Compose, Copilot is entirely **server-side** — a custom decoder-only Transformer hosted in Azure, serving 400M+ requests/day at <200ms mean latency through an elaborate global proxy infrastructure (HTTP/2 multiplexing, request cancellation, streaming, geographic routing). The model is trained specifically for Fill-in-the-Middle (FIM) completion — prefix and suffix awareness — through mid-training, supervised fine-tuning, and reinforcement learning. Copilot represents the cloud-dependent extreme of the Smart Compose paradigm: the model is too large to run on-device, so massive infrastructure is invested to make the network round-trip fast enough. Morpheus's research question sits at the opposite end: **can a Smart Compose-equivalent multi-token continuation system run entirely on-device, on a consumer laptop, for use in text editors and desktop applications?** We position our 91M model (55 MB quantized) as answering this question for the desktop paradigm — comparable in scale to Smart Compose's 80M LSTM, but running locally rather than on Cloud TPUs. For the mobile next-word prediction paradigm, a much smaller model than 91M would be required to match Gboard's 1.4 MB footprint; our current model serves as a starting point for that trajectory, and the inference engineering strategies we document (Section 5.5) are equally applicable to a future distilled mobile variant.

Trnka & McCoy (2008) defined the **Keystroke Savings Rate (KSR)** as the gold standard for word prediction evaluation: the percentage of keystrokes saved by accepting predictions. We adapt this as **Character Savings Rate (CSR)** — a simulation-based metric that does not require user studies, following the free-acceptance model where accepting a suggestion costs one keystroke (Tab). CSR measures the multi-token continuation paradigm; for next-word prediction, we introduce completion logging with replay (Section 5.5.6) as a complementary paradigm-specific evaluation.

2.2 Agglutinative Language Modeling

QuechuaTok (Contreras, 2026) introduced MorphAcc — morpheme boundary accuracy — showing that standard tokenizers radically underperform on agglutinative languages. BPE achieves fertility 1.636 but only 6.67% MorphAcc on Quechua. We adopt MorphAcc for Basque evaluation and replicate the vocabulary-size finding: our 4K Unigram tokenizer achieves 66.7% MorphAcc consistency (vs 28.6% at 32K), mirroring QuechuaTok's 4K result of 66.67%.

Plains Cree word completion (Lane et al., 2022) used a finite-state morphological analyzer

(FST) to segment Cree words into morphemes before prediction. They found that morphological segmentation is both the input representation and the evaluation target for agglutinative languages. Their KSR improvements (15-30% over non-morphological baselines) motivate future work on morphology-aware tokenization for Morpheus.

Euskarazko LLM-ak (Basque LLMs): The HiTZ center has released Latxa (Llama-2/3-based, 7B-70B) and Orai NLP has released Kimu (Gemma-2-based, 2B-9B). EvalEU benchmark (itzune.eus/evaleu, 2026) shows Kimu 9B outperforming Latxa 8B on text-relevant tasks (XNLI 74.2% vs 56.7%, EusProficiency 51.2% vs 46.3%). While these models demonstrate strong Basque language modeling, their sizes (8B-70B params) are incompatible with on-device deployment. Latxa and Kimu also inherit their parent models' tokenizers (32K and 256K respectively) without published Basque-specific tokenizer ablations — a gap our vocabulary-size experiment addresses. For baseline comparison (Section 6.7), we evaluate two HiTZ models at scales comparable to or larger than Morpheus: **HiTZ/gpt2-eus-euscrawl** (GPT-2 small, 124M, trained on the EusCrawl corpus) and **HiTZ/Latxa-Qwen3.5-2B** (1.88B, a Qwen3.5-based instruct model adapted for Basque). The former represents the small-model regime; the latter represents what a 20x larger instruct-tuned model achieves on the same evaluation corpus.

2.3 State Space Models

Mamba (Gu & Dao, 2023) introduced Selective State Space Models as an alternative to Transformers, offering linear-time inference with constant memory — no KV cache. Mamba-2 (Dao & Gu, 2024) reformulated the architecture as Structured State Space Duality, achieving 6-8x Transformer training speed with multi-head SSM support.

LFM-2.5-230M (Liquid AI, 2026) demonstrated SSM viability on edge devices: a 230M hybrid Mamba-MLP model running at 42 tok/s on a Raspberry Pi 5 and 213 tok/s on a Galaxy S25 Ultra, within 400 MB memory. This validates the on-device deployment path we pursue.

2.4 Evaluation Methodology for Autocomplete

ChaI-TeA (2024) is the closest analog to our evaluation: a large-scale benchmark for Chinese input methods with 26K+ test prefixes and a `saved@k` metric. They attempted LLM-as-judge for semantic matching of alternative completions but found it "very challenging" and deferred it — confirming the multiple-valid-completions problem we encounter.

Kosyak & Tyers (2022) evaluated predictive text for agglutinative languages using FST-based models and KSR with user studies, confirming that the multiple-valid-forms problem is central to agglutinative autocomplete evaluation.

WSTypist (2026) used simulation-based mobile typing evaluation, confirming that simulation metrics are accepted practice when user studies are infeasible, provided they are at scale with a realistic acceptance model.

---

3. Architecture Selection

3.1 Problem Constraints

Our constraints create a tight optimization problem:

Constraint	Target	Why
Inference latency	P90 <= 50ms per token	Users perceive > 100ms as lag
Memory footprint	<= 300 MB on disk, <= 500 MB RAM	Must fit consumer devices + browser extensions
No network calls	Zero external dependencies	Privacy requirement; all data stays local
Basque morphology	Correct case suffix prediction	Core user need, not just collocations
Training budget	1x NVIDIA L40 (48 GB), 5-10 days	Realistic for a small research team
Code-switching	Basque-Spanish (Euskañol)	~30% of informal Basque text contains Spanish

3.2 Candidate Architectures

We evaluated three candidate architectures against these constraints:

Path A: xLSTM (Modern Recurrent)

- 50-100M parameters
- O(1) per-step inference, constant memory, no KV cache
- Training from scratch on Morpheus Basque corpus
- ONNX Runtime INT8 deployment
- **Pros:** Safest latency, reuses v1 infrastructure, predictable
- **Cons:** No pre-trained Basque knowledge, lower expected quality ceiling

Path B: Distilled Transformer (from Kimu 9B)

- 200-500M parameters (pruned + distilled from Orai Kimu 9B)
- O(n) inference with KV cache
- Best expected quality due to pre-trained Basque knowledge
- llama.cpp GGUF deployment
- **Pros:** Highest potential quality (EvalEU-validated teacher), mature ecosystem
- **Cons:** KV cache adds latency unpredictability, tokenizer mismatch (Gemma 256K -> must prune), most engineering complexity

Path C: Mamba-2 SSM

- 100-300M parameters
- O(1) per-step inference, constant memory, no KV cache
- Parallelizable training (scan operation)
- llama.cpp GGUF deployment
- **Pros:** Best quality-latency balance, future-proof (SSMs are the on-device trajectory), LFM-2.5 validates edge feasibility

- **Cons:** Newer, less battle-tested ecosystem; ONNX support immature (use GGUF instead)

3.3 Decision: Mamba-2 (Path C)

We selected Mamba-2 for the following weighted scoring:

Criterion	Weight	xLSTM	Distilled Transformer	**Mamba-2**
Expected prediction quality	30%	3	**5**	**4**
Inference latency safety	25%	**5**	3	**5**
Engineering effort / risk	20%	**4**	2	3
Ecosystem maturity	10%	3	**5**	3
Future-proofing	10%	3	4	**5**
Training cost	5%	**5**	3	**4**
Weighted Score		3.80	3.70	**4.00**

Mamba-2 combines the LSTM-like inference properties essential for on-device autocomplete (constant memory, no KV cache, predictable latency) with dramatically better language modeling capacity than recurrent architectures at the same scale. It is the "best of both worlds" between xLSTM's latency safety and the distilled Transformer's quality potential.

This architectural choice is directly supported by Google's production experience with Smart Compose (Chen et al., 2019): despite Transformers achieving better perplexity, Google chose an LSTM for production because "the average decoding latency of Transformer is much worse than LSTM on both per-step and per-suggestion basis. This is due to keeping track of self-attention keys and values from all previous decoding steps." Google solved this latency problem with data-center Cloud TPUs; Mamba-2's O(1) per-step inference solves it at the architecture level, enabling the same Smart Compose paradigm to run **on-device** on a consumer laptop without a data center.

The decision to not pursue Path B (distillation) was driven by practical concerns: the Gemma tokenizer's 256K vocabulary would require aggressive pruning for a 200M model (embedding table alone = 524 MB), and KV cache management on consumer CPU introduces latency variance that violates our 50ms P90 constraint.

---

4. Model Architecture & Training

4.1 Model Configuration

Parameter	Morpheus-Small
Architecture	Mamba-2 (pure SSM)
d_model	768
n_layer	24
d_state	64
expand	2
headdim	64
Total params	**~91M**
Vocabulary size	4,000

Parameter	Morpheus-Small
Seq length (training)	1024
Batch size	64 (x2 accumulation)
Effective tokens/step	131,072
Learning rate	2e-3 (cosine decay to 1e-5)
Warmup tokens	50M
Total training tokens	10B
Total steps	**76,294**

At 4K vocabulary, only 3.4% of parameters are in the embedding table (3.07M of ~91M), compared to 27% at 32K. This parameter efficiency is a key advantage of the small-vocabulary approach for compact models.

4.2 Data Pipeline

Corpus: 4.62 billion subword tokens (pre-tokenized as uint16 .npy, 9.24 GB) from 11 curated Basque text sources after Phase 3 cleaning:

Source	Type	Notes
EusCrawl v2	Web crawl	Cleanest source (18.8% dup, 52.6% Basque)
HPLT v2	Web crawl	Replaced HPLT v1 (83.8% dup, 4.9% Basque — excluded)
CulturaX	Web crawl	
FineWeb2	Filtered web	
FinePDFs	PDF-extracted	
Colossal OSCAR	Web crawl	
ZelaiHandi	News/blog	
BOPV	Basque Gov. gazette	Excluded from tokenizer training (60% dup)
BOTHA	Álava gazette	Excluded from tokenizer training (64% dup)
ParlEus	Parliament transcriptions	
Wikipedia Basque	Encyclopedia	Used for validation split

Cleaning: Four-phase data cleaning pipeline applied before tokenization: 1. **Documents reparsing:** Normalize encodings, strip non-printable characters, detect corrupted documents 2. **Form regularity:** Add missing punctuation boundaries, collapse excessive whitespace, normalize line delimiters 3. **Content filtering:** Remove validation/test leakage, minimum/maximum line length filters, outlier removal 4. **Deduplication:** Corpus-level exact deduplication, near-deduplication (MinHash LSH)

Corpus-quality audit (2026-07-03): A full-source audit of the cleaned corpus (248M lines, 15 sources) revealed quality disparities. `hplt-v1` was excluded (83.80% duplicate rate, 4.9% Basque signal — replaced by `hplt-v2`). `BOG` (Gipuzkoa gazette) was excluded after an LLM-based audit found Phase 2 sentence splitting had destroyed legal text into mid-sentence fragments (2% clean

lines). `Aldizkariak` (academic journals) was excluded (35% boilerplate). `BERnaT BSM` (social media) was excluded (dialectal, non-standard orthography). The remaining 11 sources (128M lines) proceed to tokenizer and model training.

Validation leakage prevention: 68,755 lines from the held-out validation set (`wiki_valid.txt`) are excluded from training pretokenization via `--exclude-lines-file`, ensuring zero overlap between training and evaluation data.

Tokenizer: SentencePiece Unigram with **4,000-token vocabulary**, trained on 9 sources (all 11 except BOPV and BOTHA, which were excluded due to high duplicate rates that would bias the vocabulary toward gazette boilerplate). See Section 4.5 for the vocabulary-size ablation that motivated this choice.

4K tokenizer quality (verified 2026-07-04):

Metric	Result
Vocab size	4,000
Fertility (tokens/word)	2.52
Basic morphology (9 cases)	**9/9**
Multi-layer morphology (11 cases)	**11/11**
Roundtrip fidelity	100%
Character coverage	99.95%
Alphabet size	107 characters

Morphological segmentation quality: All core agglutinative patterns split cleanly: - `etxe+a` -> `|etxe a` (house + the) - `etxe+tik` -> `|etxe tik` (house + from) - `etxe+ra` -> `|etxe ra` (house + to) - `etxe+arekin` -> `|etxe arekin` (house + with) - `etxe+arentzat` -> `|etxe arentzat` (house + for) - `mendi+ra` -> `|mendi ra` (mountain + to) - `mendi+tik` -> `|mendi tik` (mountain + from)

Multi-layer morphology also resolves: `mendikoak` -> `|mendi ko ak` (mountain + of + the-plural), three distinct morphemes. Compare this to the 32K tokenizer which fused entire inflectional clusters into opaque tokens like `|etxetik`, `|etxera`, `|etxearen` — see Section 4.5 for the full comparison.

4.3 Training

Training was performed on a single NVIDIA L40 GPU (48 GB GDDR6, Ada Lovelace) with the following setup:

- **Framework:** PyTorch 2.4+ with `mamba-ssm` and `causal-conv1d` CUDA kernels
- **Precision:** bfloat16 (BF16) for training stability
- **Optimizer:** AdamW (beta_1=0.9, beta_2=0.95, weight_decay=0.1)
- **Schedule:** Linear warmup (50M tokens) -> cosine decay to 1e-5
- **GPU config:** Power limit 260W, persistence mode enabled
- **Throughput:** ~55k tok/s (settled)
- **W&B run:** `knid3x95` ("rich-forest-14"), project `morpheus-v2-mamba`

Training progress (4K model, W&B run `knid3x95`):

Milestone	Step	Valid Loss	Valid PPL	Tokens Seen	% Complete
Resume point	16,000	—	—	2.10B	21.0%
Checkpoint	32,000	2.0229	7.56	4.19B	41.9%
Mid-training	45,000	1.9864	7.28	5.90B	59.0%
Evaluation	54,000	1.9698	7.17	7.08B	70.8%
Best (converged)	**74,000**	**1.9641**	**7.13**	**9.70B**	**97.0%**
Final	76,294	1.9641	7.13	10.0B	100%

Validation loss decreased monotonically throughout training (2.05 -> 1.96), with PPL improving from 7.8 (step 25K) to 7.13 (step 74K). The model is fully converged — validation loss is flat from step 67K onward (Delta < 0.001). The best checkpoint (step 74K, valid_loss=1.9641, PPL=7.13) is used for all final evaluations and deployed models. Step 76K (final step) achieved identical PPL (7.13) and is not distinguishable in quality. The training budget of ~10B tokens yields a tokens-to-parameters ratio of approximately 110:1 — below the Mosaic inference-optimal (190:1) and MiniCPM small-model-optimal (192:1) recommendations, indicating the data budget is reasonable by modern small-model standards; see Section 6.16 for a detailed scaling-law analysis.

Checkpoint integrity: Checkpoints are saved atomically (write to `.tmp` then `os.replace()`) every 2,000 steps. A file-based stop monitor detects checkpoint completion via size-stability polling (3 consecutive unchanged polls + size >= 540MB) and sends SIGINT for clean W&B flush.

4.4 Pre-Training Validation: The Three-Gate Protocol

Perplexity is the standard language modeling metric, but it is insufficient as a sole quality gate for autocomplete: a model can achieve low PPL while producing degenerate suggestions if the training corpus contains systematic artifacts — for example, bare numbers in sentence position (decree IDs, budget amounts from official gazette texts) that bias the model toward numeric predictions instead of Basque words. This limitation is explored in detail in Section 6, where PPL improves while the multi-token autocomplete metrics lack the statistical power to confirm the improvement at current sample sizes.

To ensure data and pipeline integrity before committing GPU resources, we designed and executed a three-gate pre-training validation protocol. **No training run may proceed to the L40 without passing all three gates.**

Gate 1: Corpus Content Audit (CPU, ~30 min). An LLM-based quality audit using DeepSeek-V4-Pro evaluates 40 random lines per source, classifying each by text type, flagging quality issues, and rating Basque quality on a 1–5 scale. The 2026-07-04 audit flagged BOG (2% clean) and aldizkariak (38% clean). Both were excluded. The remaining 11 sources scored an average quality of 4.6/5.

Gate 2: Proxy Overfit Test (GPU, ~20s). A canary test: a 0.7M-parameter Mamba-2 model (128x smaller than target) attempts to memorize 5 hand-crafted Basque sentences not in the training corpus. If it can memorize novel Basque from the tokenized .npy format in 300 steps, the pipeline (tokenizer, serialization, architecture, training loop) is proven sound. Any downstream failure must then come from data quality or training scale, not infrastructure.

Metric	Value
Initial loss (step 0)	8.30 (random initialization)
Final loss (step 300)	0.049 (170x reduction)
Canary token accuracy	57.7% (>=50% threshold)
Runtime	18.5 seconds
NaN events	0

Gate 3: Autocomplete Smoke Test (GPU, ~5 min). Evaluates whether the full 91M model produces useful autocomplete suggestions on real Basque text. **Passed:** at step 32K, the model achieves CSR=24.9% on 300 held-out sentences and MorphAcc=70%, confirming the model produces coherent Basque completions.

Gate	Runtime	What it validates	clean-v3 result
1: LLM audit	~30 min CPU	Source quality, fragments, boilerplate	11/13 sources pass
2: Proxy overfit	~20s GPU	Tokenizer integrity, data format	[OK] 57.7% canary accuracy
3: Autocomplete smoke	~5 min GPU	Autocomplete quality on real text	[OK] CSR=24.9%, MorphAcc=70%

Each gate targets a distinct failure class that PPL alone cannot detect: data contamination and source quality (Gate 1), pipeline and serialization integrity (Gate 2), and inference-time autocomplete behavior (Gate 3). Together they ensure that low PPL reflects genuine language modeling quality rather than artifacts of data or infrastructure.

4.5 Tokenizer Strategy: Deep Research and Implications

The choice of tokenizer is the single most consequential design decision for agglutinative language modeling. Our tokenizer research spanned seven recent papers (2024–2026) covering 70+ languages and multiple agglutinative language families.

4.5.1 Six Papers That Reshaped Our Understanding **QuechuaTok: The fertility fallacy (Contreras, 2026).** The most directly relevant work evaluated BPE, Unigram, WordPiece, and morphology-aware PRPE on Southern Quechua — a suffixing agglutinative language structurally analogous to Basque. The key finding: **fertility rate is an insufficient, even misleading, metric.** BPE achieves the lowest fertility (1.636 at 16K) by memorizing entire polymorphemic words as single tokens, yet achieves only **6.67% MorphAcc**. PRPE achieves **83.33% MorphAcc** at competitive fertility (1.797). Crucially, **Unigram at 4K achieves 66.67% MorphAcc, dropping to 26.67% at 8K and 33.33% at 16K.** We replicate this finding for Basque (Section 4.5.4) and extend it with a stronger claim: **fertility is not just insufficient — it is a *confound* that rewards the very behavior that destroys morphological generalization.**

MorphScore-70: Alignment does not predict performance (Arnett et al., 2025). An expanded evaluation across 70 languages and 5 pre-trained tokenizers found morphological alignment

explains only a small fraction of variance (R^2 = 0.005–0.024) in downstream performance. For **Basque specifically**, MorphScore precision was 0.11–0.12 across all tokenizers — among the lowest of 70 languages.

Uralic tokenization: Unigram > BPE (Xu & Kim, 2026). A controlled comparison across six Uralic languages found Unigram consistently outperforms BPE on POS tagging for agglutinative languages, especially in low-resource settings.

MorphPlaus: Evaluating without gold data (Stephen & Libovický, 2026). An IBM Model 1 alignment metric correlates strongly with boundary recall (rho > 0.70). Confirms that Unigram > BPE > WordPiece for morphological alignment, and smaller vocabularies yield better alignment.

Morphology-aware tokenization for Spanish (García et al., 2025). Training BPE on morphologically pre-segmented text improved masked LM performance. Pre-segmentation with a morphological analyzer before tokenizer training produces linguistically meaningful subwords.

Tokenization for Turkish/Finnish (Hu, 2025). Under low-resource conditions, word-level tokenization consistently outperformed BPE for morphologically rich languages.

4.5.2 What Basque LLMs Chose **Latxa (HiTZ, 2024):** The dominant Basque LLM family (7B–70B) uses Llama 2's BPE tokenizer (32K vocabulary) without modification. The Latxa paper does not discuss tokenizer choices.

Kimu (Orai NLP, 2025): The Gemma-2-based Basque model family inherits Gemma's SentencePiece tokenizer (256K vocabulary).

The open question: Neither Latxa nor Kimu has published an ablation study on tokenizer impact for Basque. At 7B-70B parameters, models overcome tokenizer deficiencies through scale — but our 91M model cannot afford a suboptimal tokenizer.

4.5.3 Retrospective on Our Decision Our v1 tokenizer decision chose SentencePiece Unigram at 32K based on fertility 1.71. The 2026 research reveals:

1. **Unigram was correct over BPE.** Xu & Kim (2026) and Stephen & Libovický (2026) support Unigram for agglutinative settings.
2. **32K vocabulary places us in the surface-form memorization regime.** At 32K, the tokenizer memorizes frequent wordforms as atomic units, fragmenting morphemes arbitrarily.
3. **No morphological pre-segmentation was the critical omission.** The literature converges: high morpheme-boundary accuracy requires injecting morphology into tokenizer training.

4.5.3.1 The Morfessor Attempt and Pivot Our first approach to improving morphological alignment was **morphological pre-segmentation** — training a Morfessor 2.0 model on the corpus, using it to segment words into morphemes before tokenizer training, then training a Unigram tokenizer on the pre-segmented text. This followed v1's ADR-002 ("Morfessor Pre-Segmentation as Mandatory Pre-Processing Step"), which had been accepted but never executed.

We trained Morfessor 2.0 on 4M words from a corpus sample. The segmentation quality was **poor** due to the mixed-language nature of the corpus (Basque + Spanish + English):

Input	Morfessor output	Problem
cookie	coo|kie	English word segmented incorrectly
dutenez	duten|ez	Should be dute|nez or unsplit

Morfessor's MDL objective fails on mixed-language text without language filtering: it cannot distinguish Basque morphology from arbitrary character sequences in foreign words. Language-filtering the corpus before Morfessor was deemed too costly for the expected benefit.

Pivot: Rather than fix Morfessor, we noticed that the QuechuaTok paper showed vocabulary size alone drives significant MorphAcc gains (Unigram 4K = 66.67% **without** any pre-segmentation). We pivoted to testing vocabulary size as the primary variable — a simpler experiment with a stronger literature basis. Morfessor pre-segmentation remains unexplored and is deferred to future work with Apertium (Section 7.2), which provides linguistically grounded morpheme boundaries rather than statistical guesses.

4.5.4 The Vocabulary Ablation Experiment We formulated a testable hypothesis:

> The morphological boundary accuracy of a SentencePiece Unigram tokenizer degrades monotonically with vocabulary size because larger vocabularies accumulate frequent surface forms as atomic tokens, fragmenting morpheme boundaries arbitrarily.

Metric: MorphAcc consistency. For each test word with a known root–suffix boundary (e.g., `etxe|tik`), we check whether the tokenizer places a token boundary at the morpheme boundary. A word scores `boundary_correct = true` if and only if the root and suffix are in **separate tokens** — not merely substrings of a single fused token. For example, `etxetik -> |etxe tik` scores [OK] (boundary preserved); `etxetik -> |etxetik` scores [X] (fused into one token). The aggregate metric is the percentage of test words with `boundary_correct = true`.

Experimental design. We trained SentencePiece Unigram tokenizers at 4K, 8K, 16K, and 32K on a proportional sample from the full corpus (336.8 MB, 3.5M lines, ~84M tokens, drawn proportionally from all 15 source files). All tokenizers used identical SentencePiece parameters: `model_type=unigram`, `character_coverage=0.9995`, `byte_fallback=True`. Training took ~60s per tokenizer on CPU. The 32K baseline was our existing production tokenizer trained on the same corpus with the same parameters.

Test words. We evaluated 21 Basque words covering five roots (`etxe`, `lagun`, `mendi`, `gizon`, `kale`) and eight case suffixes (absolutive `-a`, allative `-ra`, ablative `-tik`, genitive locative `-ko`, inessive `-an`, comitative `-arekin`, benefactive `-arentzat`, causal `-arengatik`). Representative examples:

Root	Suffix	Test word	Expected boundary
etxe	-a	etxea	etxe|a
etxe	-ra	etxera	etxe|ra
etxe	-tik	etxetik	etxe|tik
etxe	-ko	etxeko	etxe|ko
etxe	-arekin	etxearekin	etxe|arekin
etxe	-arentzat	etxearentzat	etxe|arentzat
mendi	-ra	mendira	mendi|ra
mendi	-ko	mendiko	mendi|ko

Root	Suffix	Test word	Expected boundary
kale	-tik	`kaletik`	`kale|tik`
kale	-an	`kalean`	`kale|an`

(The full 21-word list with per-tokenizer results is available in the supplementary materials.)

Results:

Tokenizer	Vocab Size	Fertility	MorphAcc Consistency	Change
`baseline-32k`	32,000	1.85	**28.6%** (6/21)	baseline
`raw-16k`	16,000	2.06	**52.4%** (11/21)	+23.8pp
`raw-8k`	8,000	2.28	**61.9%** (13/21)	+33.3pp
`raw-4k`	4,000	2.58	**66.7%** (14/21)	**+38.1pp**

The hypothesis was confirmed with striking precision:

1. **The QuechuaTok finding replicates for Basque.** The 4K MorphAcc (66.7%) is nearly identical to QuechuaTok's 4K Unigram result (66.67%).
2. **The degradation is monotonic and accelerating.** Each doubling of vocabulary costs progressively more: 4K->8K loses 4.8pp, 8K->16K loses 9.5pp, 16K->32K loses 23.8pp.
3. **Fertility is the mechanism, not the metric.** The 4K tokenizer has the worst fertility (2.58 vs 1.85 for 32K) because it is forced to decompose — and that forced decomposition is precisely what preserves morpheme boundaries.
4. **4K is the sweet spot for Basque at 91M parameters.** At 4K, every root is a separate token from every case suffix, and verbal agreement morphemes are independently accessible.

4.5.4.1 Beyond Nominal Morphology: The Verbmorph Gap The MorphAcc consistency metric in Section 4.5.4 tests **nominal morphology** — root + case suffix. The decisive difference between 4K and 32K emerges even more starkly in **verbal morphology**, where Basque's polysynthetic verb structure encodes subject, object, indirect object, tense, and mood in a single word:

Verb form	Gloss	4K tokenization	32K tokenization
`dizkizut`	*I have them to you*	`|di zki zu t`	`|dizkizut`
`dakizkioke`	*he can know them to him*	`|da ki zki o ke`	`|dakizki ok e`
`zitzaizkidan`	*they were to me*	`|zitzai zki dan`	`|zitzaizkidan`

At 4K, the pluralizer `zki` is an independent reusable token present in all three verbs. A Mamba-2 model can productively recombine `di`, `zki`, `zu`, `t` to form unseen verb inflections. **At 32K, `zki` is buried inside opaque atomic tokens** that share no subword structure.

This is the **fertility paradox**: lower fertility (fewer tokens per word) is achieved exactly by fusing morphemes into opaque units. The fertility metric favors the very behavior that destroys morphological generalization. **Low fertility is the *mechanism* of the surface-form memorization regime, not a desirable property.**

4.5.4.2 Where 4K Still Fails The 4K tokenizer is not perfect. It handles single-layer case suffixes well but struggles with two categories:

Word	4K tokenization	Expected	Problem
`etxean`	`|etxean`	`etxe|an`	Whole word not split (inessive)
`etxearentzat`	`|etxe a rentzat`	`etxe|arentzat`	Multi-layer suffix split incorrectly
`etxearengatik`	`|etxe aren gatik`	`etxe|arengatik`	Multi-layer suffix split incorrectly
`lagunera`	`|lagun era`	`lagun|ra`	Epenthetic `e-` fused with suffix
`lagunetik`	`|lagun etik`	`lagun|tik`	Epenthetic `e-` fused with suffix

Two failure modes: 1. **Multi-layer suffixes** (`a+ren+tzat`): when a case suffix itself decomposes into multiple morphemes, 4K sometimes splits at the wrong internal boundary. 2. **Epenthetic vowels**: Basque inserts `e-` before consonant-initial suffixes after certain roots (`lagun` + `tik` -> `lagunetik`). The 4K tokenizer fuses this epenthetic vowel with the suffix (`lagun` + `etik`) rather than with the root.

These remaining failures motivate the Apertium pre-segmentation future work (Section 7.2): a morphological analyzer that provides explicit, linguistically grounded boundaries would resolve both multi-layer suffixes and epenthetic vowel placement — problems that vocabulary size alone cannot fully solve.

4.5.5 Downstream Perplexity Confirmation The vocabulary ablation was further validated by downstream perplexity evidence from the QuechuaTok study: 4K Unigram achieves the lowest downstream perplexity among vocabulary sizes tested, confirming that the morphological alignment advantage translates to better language modeling, not just better MorphAcc.

At our model scale, the parameter-efficiency argument is also decisive: at 4K vocab, only 3.4% of the 91M parameters are embeddings (vs 27% at 32K), freeing capacity for the SSM layers that drive sequence modeling quality.

---

5. Deployment Pipeline

5.1 Export: PyTorch -> HuggingFace -> GGUF

Direct ONNX export of Mamba models is not viable for production CPU inference. Instead, we use `llama.cpp`'s native Mamba-2 support:

```
1. PyTorch checkpoint -> HuggingFace format (custom export script)
2. HuggingFace -> GGUF FP16 (llama.cpp convert_hf_to_gguf.py)
3. GGUF FP16 -> Q4_K_M quantization (llama-quantize)
```

The export script writes `tokenizer_config.json` with `add_bos_token: false` and a `generation_config.json`, ensuring llama.cpp does not auto-prepend a BOS token. This fix was motivated by a debugging experiment: the model was trained without BOS, but llama-server auto-prepended BOS for string prompts, producing CSR of ~4% (vs ~28% with correct token-ID prompts). This was diagnosed as two compounding issues — (1) BOS mismatch (the model never saw BOS during training) and (2) llama.cpp's built-in SentencePiece tokenizer diverging from the reference `sentencepiece` library on this 4K vocabulary, causing incorrect tokenization of long words. The fix is two-layer: the export writes `add_bos_token=false`, and the demo sends token IDs (not strings) to llama-server (Section 5.4).

5.2 Quantization

Format	Size	BPW	Notes
FP32 (PyTorch checkpoint)	~547 MB	32	Training only
FP16 (GGUF)	**181 MB**	16	Reference quality
Q8_0 (GGUF)	97 MB	8	Near-lossless
Q5_K_M (GGUF)	64 MB	5.5	High quality
Q4_K_M (GGUF)	**55 MB**	4.92	**Deployment default**

5.3 Inference

On the NVIDIA L40 (GPU), the f16 model generates at ~550 tok/s in batch. On consumer x86-64 CPU (Intel Xeon, 8 threads), the Q4_K_M quantized model achieves ~250 tok/s in batch inference via llama-server. Interactive single-token generation is estimated at ~40-80 tok/s.

5.4 Demo Server

A Docker-based demo server provides: - **Greedy mode** (Smart Compose style): temperature=0.0, deterministic suggestions - **Sampling mode**: configurable temperature, top_p, WebSocket streaming - **Token-ID prompts**: The demo sends token IDs (not string prompts) to llama-server. This was motivated by a debugging experiment: string prompts produced CSR of ~4% (vs ~28% with token-ID prompts). Two compounding issues were diagnosed: (1) llama-server auto-prepended BOS for string prompts (the model was trained without BOS), and (2) llama.cpp's built-in SentencePiece tokenizer diverges from the reference `sentencepiece` library on this 4K vocabulary, producing incorrect tokenization of long words. Sending token IDs (encoded with the reference library) bypasses both issues, ensuring inference matches training semantics exactly. - **Smart context**: strips trailing subword fragments so the model sees only complete tokens - **Ghost suffix**: deduplication of user-typed text from model prediction for inline ghost text display - **Output filtering**: punctuation collapse, U+FFFD removal (for undertrained model artifacts) - **Model hot-reload**: `POST /api/model/reload` swaps GGUF files without container restart

5.4.1 Two Prediction Paradigms

The demo implements two distinct prediction paradigms, which we distinguish throughout the evaluation because they have different user experiences, failure modes, and applicable metrics:

	Multi-token continuation	**Next-word prediction**
Metaphor	Smart Compose (Gmail)	Predictive keyboard (smartphone)
Production reference	Smart Compose: ~80M LSTM, **server-side** (Cloud TPU)	Gboard: 1.4M LSTM, 1.4 MB, **on-device**
Morpheus model	91M (55 MB Q4_K_M), **on-device** (consumer CPU)	91M — too large for mobile; needs distillation to match Gboard's 1.4 MB
Output	N tokens of gray inline text	3 discrete word chips
Acceptance	Tab accepts the entire continuation	Tap selects one word
Decoding	Greedy, multi-token (up to 15)	Greedy, word-level with fallback
Failure mode	Repetition loops, ghost-text jitter	Tokenization trap, prediction vanishing
Evaluated by	CSR, Human A/B (Section 6)	Completion logging + replay (Section 5.5.6)

These paradigms share the same underlying model and tokenizer but require different inference engineering and target different deployment contexts. **Multi-token continuation** is the primary research question of this work: can a Smart Compose–equivalent system run entirely on-device, on a consumer laptop, for text editors? Our 91M model (comparable to Smart Compose's 80M) answers this affirmatively — but runs locally rather than on Cloud TPUs. **Next-word prediction** is the mobile keyboard paradigm, where Gboard's 1.4M/1.4MB model defines the deployment target. Our 91M model is too large for that context; the strategies documented in Section 5.5 are the inference engineering that would carry over to a future distilled mobile model.

Multi-token continuation is the simpler inference case: the model greedily extends the context and the result is displayed inline. Next-word prediction is harder because it must produce *whole words* as discrete options, which exposes the tokenization trap (Section 5.5.1): the subword path the user's partial input lands on may not reach the correct word. The strategies in Section 5.5 address this second paradigm specifically.

This distinction also maps to our evaluation: CSR (Section 6.3) and the human A/B evaluation (Section 6.6) measure multi-token continuation quality, while completion logging (Section 5.5.6) measures next-word prediction quality. Conflating the two would obscure why, for example, PPL improvements are not confirmed by CSR (a multi-token, exact-match metric with overlapping CIs) while the keyboard experience benefits from candidate carry-forward (a next-word strategy) and the next-word replay shows 54K ahead (Top-1 60->80%).

5.5 Inference Engineering for Agglutinative Keyboards

This section addresses the **next-word prediction** paradigm (Section 5.4.1): deploying a language model as a real-time predictive keyboard that offers whole-word suggestion chips. This is the mobile keyboard paradigm exemplified by Gboard (1.4M params, 1.4 MB on-device). Our current 91M model (55 MB quantized) is sized for the desktop multi-token continuation use case and is too large for a production mobile keyboard; however, the inference engineering strategies documented here —

retokenization fallback, sticky merge, top-k alternatives, next-word extraction — are architecture-agnostic and would carry over directly to a future distilled mobile model. We develop and validate them using the 91M model because it is large enough to produce meaningful predictions on which to observe and fix the failure modes unique to agglutinative next-word prediction. This is the harder of the two paradigms because it must produce discrete, complete words — which exposes the tokenization trap, a structural failure mode of subword tokenization that does not appear in multi-token ghost-text continuation or in batch evaluation. Each strategy below is motivated by a concrete failure observed during development.

5.5.1 The Tokenization Trap In an agglutinative language with a 4K subword vocabulary, the same word can be reachable through multiple token paths — and the path the user's partial input lands on may not reach the correct completion.

Example: The Basque word *Kaixo* ("hello") tokenizes as `[|Ka, i, xo]`. But when the user types *Kaix*, the tokenizer segments it as `[|Ka, ix]` — a different path that cannot reach the `xo` token. The model may know the word perfectly, but the greedy continuation from `[|Ka, ix]` produces *Kaixan*, *Kaix-*, *Kaixko* — never *Kaixo*.

This is not a model deficiency; it is a structural artifact of subword tokenization. The problem is especially acute in agglutinative languages because long, morphologically complex words have many possible segmentation paths, and short prefixes often land on the wrong one.

5.5.2 Retokenization Fallback **Strategy:** When generating word-completion candidates, query the model from progressively shorter prefixes in parallel, then filter results by the user's actual typed prefix.

For input *Kaix*, the system queries three paths simultaneously:

Path	Prefix	Token IDs	Can reach *Kaixo*?
0	`Kaix`	`[|Ka, ix]`	[X] (wrong path)
1	`Kai`	`[|Ka, i]`	[OK] (`xo` is reachable)
2	`Ka`	`[|Ka]`	[OK] (but noisier)

Path 1 reaches `[|Ka, i]`, where the model predicts `xo` at 54.2% probability. The result *Kaixo* passes the `startswith("Kaix")` filter and surfaces as a candidate. All paths fire in parallel via `asyncio.gather`, keeping latency at ~1x a single call rather than 3x.

A **from-scratch path** (empty prefix, predicting the next word from preceding context only) rescues single-token whole words. For example, *bezala* is a single token (`|bezala`); when the user types *b*, the token `|b` cannot reach `|bezala`. Querying from the preceding context (*"Ni ondo, beti"*) surfaces *bezala* as a next-word prediction, which passes the `startswith("b")` filter.

5.5.3 Sticky Merge: Candidate Carry-Forward **Problem:** When the model predicts a next word (e.g., *izan* after *idatzia*) and the user types the first letter (*i*), the system switches from next-word prediction to word-completion mode. The token path changes (`|izan` is one token, but `|i` + continuation is a different path), and the previously good prediction vanishes from the candidate list — even though the user is typing exactly the word that was predicted.

Strategy: Maintain a *sticky pool* of the previous render's candidates. When new candidates arrive, filter the sticky pool by the current typed prefix. Survivors are merged with fresh candidates, receiving a small probability boost (+0.1) to compensate for the fact that cross-path probabilities (next-word vs. word-completion) are not directly comparable.

```
State 1: "...idatzia" -> [dago, dezakezu, daiteke, behar, izan]
  (izan at rank 5, not visible in top-3 chips, but stored in sticky pool)

State 2: "...idatzia i" -> fresh: [iristeko, itxa, ikur, ...]
  Sticky survivor: izan (prob=0.087, boosted to 0.187)
  Merged result: [iristeko, izan [OK], itxa]
```

The sticky pool resets on chip acceptance and message send, preventing stale candidates from persisting across word boundaries.

5.5.4 Top-k Exceeds Display-k The keyboard displays 3 suggestion chips but fetches 5 candidates from the server. The extra candidates populate the sticky pool, enabling carry-forward of lower-ranked but relevant predictions. Without this, *izan* (rank 5, prob=0.087) would never enter the sticky pool and could not be rescued in State 2.

5.5.5 Next-Word Candidate Extraction When the model's greedy continuation at a word-completion level begins with a space (|-prefixed token), it signals that the model considers the current word complete and is predicting the *next* word. Rather than discarding these tokens (as noise), we extract them as next-word candidates with an `is_next_word` flag. The frontend handles these differently: inserting a leading space before the word, matching the user's expectation that the current word is finished.

This also handles the edge case where the user has typed a complete word without a trailing space (e.g., *"Kaixo, zer"*): the model predicts *moduz* as a next word, and the candidate appears despite no explicit word boundary.

5.5.6 Completion Logging and Replay Every chip acceptance is logged to a JSONL file with: timestamp, model checkpoint, context, smart context, accepted word and its probability, and all candidates offered. This transforms real user sessions into an evaluation dataset that can be replayed against any checkpoint:

```
python scripts/replay_completions.py --models step_0032000.Q4_K_M step_0054000.Q4_K_M
```

The replay script hot-reloads each checkpoint, queries the same contexts, and checks whether the user-accepted word appears in the top-k. This enabled a critical finding: an apparent model regression (step 54K "forgetting" *Kaixo*) was diagnosed as an **inference engine bug**, not a model deficiency (Section 5.5.7).

The keyboard candidate algorithm (retokenization fallback, sticky merge, top-k fetch, acceptance semantics) is also ported to PyTorch in `src/eval_utils.py` as `evaluate_next_word_csr`, enabling training-time validation that faithfully reflects the deployed demo. This runs natively on the GPU model (no llama.cpp dependency) during periodic validation, reporting decomposed metrics (word accuracy, acceptance rate, average prefix length, average confidence) alongside a simulated CSR. It is used as a **secondary metric** — PPL remains primary for checkpoint ranking — and the

decomposed metrics avoid the CSR paradox (Section 6.14) because they do not conflate model quality with morphological word length.

5.5.7 Inference Engine Sensitivity During development, step 54K appeared to have "forgotten" the word *Kaixo* (predicting it at 0% probability) while step 32K predicted it correctly at 47.5%. Investigation via the replay system revealed that the model was not at fault: a stale Docker cache had built an older version of `llama.cpp` that contained a bug in the SSM (State Space Model) scan computation for Mamba-2 architectures (`n_groups > 1`, fixed in commit `dc2187d48`). The bug produced silently incorrect greedy outputs for certain weight configurations — step 54K's weights triggered it, step 32K's did not.

Recommendation: When deploying Mamba-2 models with `llama.cpp`, pin to a build that includes commit `dc2187d48` (2025-07-04 or later). If a checkpoint appears to have regressed, rebuild the inference engine with `--no-cache` before investigating the model. The completion logging and replay system (Section 5.5.6) is invaluable for detecting such issues, as it enables direct A/B comparison of checkpoints on identical inputs.

---

6. Evaluation

6.1 Evaluation Methodology

We employ a multi-metric evaluation suite designed to capture different aspects of autocomplete quality for agglutinative languages. Each metric has known limitations, and we use them in concert to triangulate true model quality.

Metrics

1. **Perplexity (PPL)** — The full next-token distribution quality. Computed on 1.83M held-out tokens (the validation set excluded from training via line-level leakage prevention). This is the smoothest, lowest-variance metric with no exact-match artifact. Computed with training-matching semantics: 1024-token windows, `</s>` separators included in loss, `ignore_index=0` for `<unk>`, no BOS, bfloat16.

2. **Character Savings Rate (CSR)** — Simulates keystroke-by-keystroke typing. For each character in the target completion, the model receives `prompt + typed_so_far` and we check if its top-1 greedy prediction aligns with the remaining target. Acceptance costs 1 keystroke (Tab), following Trnka & McCoy (2008). We report **bootstrap 95% confidence intervals** (1000 resamples) on 300 held-out sentences. *This metric measures the **multi-token continuation** paradigm (Section 5.4.1): the model produces a greedy continuation and we check character-level alignment.*

3. **Morpheme Boundary Accuracy (MorphAcc)** — For test cases with known morphological segmentation (e.g., `etxe|tik`), we compute whether the model's top-5 predictions include a token that respects the morpheme boundary. 50 tests across 5 roots x 4 case suffixes, with and without context.

4. **Case Paradigm Completion** — For each of 6 Basque nouns, test all 14 grammatical cases (84 total). The model receives the bare root and we check if the correct case suffix ranks in top-K.

5. **Blinded Human A/B Evaluation** — 30 fresh prompts from held-out validation text. Both checkpoints generate greedy completions (f16, reference sentencepiece, no quantization confound). A/B randomly assigned per prompt. The Basque expert judges quality (grammaticality, naturalness, autocomplete usefulness) without knowing which checkpoint is which. *This measures the **multi-token continuation** paradigm.*

6. **Completion Logging + Replay** — Real user chip acceptances are logged with full candidate context and replayed across checkpoints (Section 5.5.6). *This measures the **next-word prediction** paradigm.*

7. **Typing Simulation (CSR Paradox)** — A frontend-faithful simulation (sticky merge, top-3 chips, acceptance semantics) types 15 translated sentences (5 Basque, 5 English, 5 Spanish) char-by-char, accepting suggestions the moment they match. Measures word accuracy, acceptance rate, confidence, and prefix length before acceptance. *This measures the **next-word prediction** paradigm and reveals the CSR paradox (Section 6.14).*

8. **Next-Word CSR (training-time)** — A PyTorch-native port of the demo keyboard algorithm (retokenization fallback, sticky merge, top-3 display, acceptance semantics) that runs during training validation on the same 30 CSR test sentences. Reports decomposed metrics: word accuracy, acceptance rate, average prefix before acceptance, and average confidence — alongside a simulated CSR. *This is a **secondary metric**; PPL remains primary for checkpoint ranking. The decomposed metrics avoid the CSR paradox (Section 6.14) because they do not conflate model quality with morphological word length.*

9. **Bits Per Character (BPC)** — Total negative log-likelihood in bits divided by character count. Unlike per-token PPL, BPC is **tokenizer-independent**: it normalizes across different vocabulary sizes, enabling fair comparison between models with 4K, 50K, and 248K vocabularies. Used for the cross-model baseline comparison (Section 6.7). Each model tokenizes the evaluation corpus with its own tokenizer, and BPC is computed as `total_nll_bits / total_chars`.

10. **Simplified Next-Word CSR (cross-model)** — A stripped-down version of the next-word CSR protocol with no inference engineering (no retokenization fallback, no sticky merge, no top-k alternatives). The model greedily decodes until a word boundary, and the first generated word is compared to the target. Used for fair raw-model comparison across architectures (Section 6.7).

Why multiple metrics No single metric is sufficient for agglutinative autocomplete. In practice, however, we found that **PPL is the only metric that consistently produces coherent, reliable signal** across our experiments. The autocomplete-specific metrics (CSR, human A/B) proved difficult to implement correctly and yielded weak or noisy data, for reasons we detail in Section 6.8–6.9:

- **PPL** measures distribution quality with high statistical power (1.83M tokens, 14 files, all agreeing) and is the only metric that unambiguously ranked the two checkpoints. Its limitation is that it does not measure autocomplete utility directly.
- **CSR** measures keystroke savings but uses exact-match gold, which cannot credit valid Basque alternative continuations (the agglutinative multiple-valid-forms problem). It requires careful implementation (token-ID prompts, no BOS, reference sentencepiece) and even then produces overlapping confidence intervals at n=300 that cannot distinguish competent models.

- **Human A/B** is theoretically the ground truth but is expensive, sample-limited (n=30), and subject to bias if not blinded. At n=30, a binomial test yields p=0.82 — insufficient statistical power to distinguish models even if a real difference exists.
- **MorphAcc** measures morphological competence but not end-to-end utility
- **Completion logging + replay** captures real next-word usage but is built from a small number of sessions and reflects inference engineering as much as raw model quality

6.2 Perplexity (PPL)

PPL is computed on two text sets: (1) the held-out validation set (1.83M tokens, genuinely excluded from training), and (2) a real corpus of Wikipedia + Berria articles (140K tokens, **contaminated** — these articles appear in training sources, so absolute PPL is optimistic; the relative comparison remains valid as all checkpoints saw the same text).

Metric	Step 32K	Step 54K	Step 74K	Trend
Held-out valid PPL (clean)	7.56 (loss 2.0229)	7.17 (loss 1.9698)	**7.13** (loss 1.9638)	↓ monotonic
Real corpus PPL (contaminated)	10.53 (loss 2.3540)	9.90 (loss 2.2923)	**9.83** (loss 2.2853)	↓ monotonic

Per-file consistency: Every single one of the 14 real-corpus files shows monotonic improvement across all three checkpoints, with 32K->54K per-file deltas ranging from -0.54 to -0.89 PPL points and 54K->74K deltas ranging from -0.03 to -0.10. The 54K->74K improvement is small — the model has essentially converged — but it is consistent across all files with no reversals. This is statistically unambiguous.

> **Note:** PPL is not comparable across different vocabulary sizes. The old 32K-vocab model reached PPL=30.78 at step 30K, but this number is not directly comparable to the 4K-vocab model's PPL=7.56 because the vocabulary size affects PPL (smaller vocab = fewer choices per token = lower PPL). All comparisons here use the *same* 4K vocabulary, so they are valid.

6.3 Character Savings Rate (CSR)

Definition:

$$CSR = 1 - \frac{\text{keystrokes\_needed} + \text{acceptance\_cost}}{\text{len(target\_completion)}}$$

where acceptance_cost = 1 (Tab key to accept the suggestion), following the free-acceptance model of Trnka & McCoy (2008).

Results (300 held-out sentences from `wiki_valid.txt`, seed=20260710, f16 GPU inference):

Metric	Step 32K	Step 54K	Step 74K
Macro CSR	24.90%	25.23%	**25.26%**
Micro CSR	24.57%	25.03%	25.06%
95% CI (macro)	[23.64%, 26.21%]	[23.98%, 26.48%]	[24.00%, 26.52%]
n_tests	300	300	300

All confidence intervals overlap -> the differences are NOT statistically significant. 74K is directionally the best (agreeing with PPL), but CSR cannot distinguish the three checkpoints at this quality level. Notably, the 54K->74K improvement in CSR (+0.03pp) is negligible despite continued PPL improvement (7.17->7.13), confirming that CSR saturates once models reach competence.

By target length — where the improvement lives:

Category	Step 32K	Step 54K	Step 74K
Short (4-6 words)	26.87%	26.88%	26.72%
Medium (7-9 words)	24.15%	24.24%	24.61%
Long (10-12 words)	21.54%	22.92%	22.87%

Short completions are saturated. The 32K->54K improvement concentrates in long targets (+1.39pp), but this gain does not continue to 74K (22.92% -> 22.87%, effectively flat). CSR has saturated.

6.4 Morpheme Boundary Accuracy (MorphAcc)

Results (50 tests, 5 roots x 4 suffixes x 2 conditions [bare + contextual], f16 GPU):

Metric	Step 32K	Step 54K	Step 74K
MorphAcc	70% (35/50)	**76%** (38/50)	**76%** (38/50)
Avg boundary prob mass	17.8%	19.4%	19.5%

MorphAcc improved from 32K to 54K (+6pp) but plateaus at 54K — step 74K shows no further improvement (76%, same 38/50 hits). The average boundary probability mass continues to creep up slightly (19.4% -> 19.5%), suggesting marginally more confident morphological predictions, but the hit count has saturated. This is consistent with the tokenizer-bound nature of MorphAcc: once the model learns the morpheme boundaries that the 4K tokenizer makes available, further training cannot improve MorphAcc without a better tokenizer (Section 7.2).

Context matters: Bare nouns (`etxe`, `mendi`) produce probability distributions dominated by punctuation and fragments — the model needs syntactic context to predict case suffixes. With full sentence context (`Bihar...nire etxe -> ra`), the model correctly places significant probability mass on the correct suffix. This confirms the Mamba-2 architecture *can* learn morphology — it just needs sufficient context and training.

Comparison to old 32K-vocab model: The old 32K-vocab model at step 30K achieved only 20% MorphAcc (1/5 tests). The 4K-vocab model's 70-76% MorphAcc represents a dramatic improvement, validating the vocabulary-size ablation (Section 4.5.4).

6.5 Case Paradigm Completion

Results (84 tests: 6 roots x 14 cases, f16 GPU):

Metric	Step 32K	Step 54K	Step 74K
Paradigm Hit@1	13.1% (11/84)	13.1% (11/84)	10.7% (9/84)
Paradigm Hit@3	20.2% (17/84)	21.4% (18/84)	22.6% (19/84)
Paradigm Hit@5	28.6% (24/84)	27.4% (23/84)	27.4% (23/84)

The paradigm metrics are noisy across checkpoints — Hit@1 actually *drops* from 13.1% to 10.7% between 54K and 74K, while Hit@3 *improves* from 21.4% to 22.6%. Hit@5 is flat. This noise further confirms that the autocomplete-specific metrics do not reliably track model quality: the paradigm test is high-variance (84 tests, bare-root prompts) and small improvements in PPL do not produce monotonic improvements in case suffix prediction. The absolutive case (-a, the citation form) is well-learned (83% Hit@5 at all checkpoints). The ergative (-ak) and inessive (-an) show partial learning. Most other cases remain below threshold — morphology emerges late and requires more context than bare-root prompts provide.

6.6 Blinded Human A/B Evaluation

Methodology: 30 fresh prompts from held-out validation text (zero overlap with CSR held-out set). Both checkpoints generate greedy completions (15 tokens max, f16, reference sentencepiece, no BOS, no quantization confound). A/B randomly assigned per prompt (different mapping in each batch). The Basque expert judges quality without knowing which checkpoint is A or B.

Results:

	Step 32K	Step 54K	Ties
Batch 1 (20 prompts)	7	6	7
Batch 2 (10 prompts)	4	3	3
Combined (30 prompts)	**11 (55%)**	**9 (45%)**	**10 (33%)**

Binomial test (two-sided): p = 0.82 — definitive statistical tie. An 11-vs-9 split out of 20 decisive judgments happens ~41% of the time by pure chance. The two checkpoints are statistically equivalent in autocomplete quality.

Expert qualitative observations: 1. **Repetition loops** — both models fall into them on some prompts (greedy-decoding failure mode, not checkpoint-specific) 2. **"Both could be correct"** — the expert noted this on ~10/30 prompts, confirming the agglutinative multiple-valid-forms problem that caps CSR 3. **"Hard to judge without the full sentence"** — autocomplete quality is context-dependent and genuinely ambiguous

6.7 Cross-Model Baseline Comparison

To contextualize Morpheus's performance, we evaluated two external Basque language models under the same evaluation protocol: **HiTZ/gpt2-eus-euscrawl** (GPT-2 small, 124M parameters, trained on the EusCrawl corpus, ~423M tokens) and **HiTZ/Latxa-Qwen3.5-2B** (Latxa, 1.88B parameters,

a Qwen3.5-based instruct model adapted for Basque). All three models were evaluated on the same corpus (`eval/real_corpus/`, 475,750 characters across 14 Wikipedia and Berria news files) and the same 30-sentence CSR test set.

BPC: The Correct Cross-Model Metric Per-token perplexity (PPL) is tokenizer-dependent and cannot be compared across models with different vocabulary sizes. A model with a 4K vocabulary produces more, shorter tokens than one with 50K, inflating per-token PPL without reflecting actual prediction quality. **Bits Per Character (BPC)** normalizes this: it computes the total negative log-likelihood in bits and divides by the number of characters, making it independent of tokenization choices.

Model	Params	Vocab	BPC	PPL (token)	Tok/Char
GPT-2 eus-euscrawl	124M	50K	0.981	29.21	0.202
Morpheus v2 (Mamba-2)	**91M**	**4K**	**0.970**	**9.83**	**0.294**
Latxa-Qwen3.5-2B	1,882M	248K	0.822	4.89	0.359

> **WARNING: Corpus contamination caveat:** Wikipedia and Berria articles may appear in all three models' training data. Absolute BPC values are optimistic; the relative comparison is valid.

Key findings:

1. **Morpheus outperforms GPT-2 on BPC** (0.970 vs 0.981) despite having 27% fewer parameters (91M vs 124M). The primary driver is training data volume: Morpheus was trained on ~10B tokens vs GPT-2's ~423M tokens — a 24x difference. This confirms that for low-resource languages, training data quantity matters more than parameter count. However, the near-tie in BPC despite the 24x data difference also reveals that character-level metrics saturate before morphological competence is achieved — see Section 6.16 for a detailed data scaling analysis.
2. **Latxa achieves the lowest BPC** (0.822), as expected for a 1.88B-parameter model — 20x larger than Morpheus. However, the BPC gap (0.148 bits/char) is modest relative to the parameter difference, reflecting diminishing returns from scale.
3. **PPL is misleading across vocabularies.** GPT-2's per-token PPL of 29.21 appears catastrophic compared to Morpheus's 9.83, but BPC reveals the models are nearly equivalent (0.981 vs 0.970). The apparent PPL gap is almost entirely an artifact of GPT-2's 50K vocabulary producing longer tokens that are individually harder to predict.

Simplified Next-Word CSR (No Inference Engineering) To provide a fair raw-model comparison without our inference engineering advantages, we evaluated all three models with a simplified next-word CSR protocol: greedy decode until a word boundary, extract the first word, and compare to the target word.

Model	CSR (macro)	95% CI	Word Accuracy
GPT-2 eus-euscrawl	0.110	[0.060, 0.167]	37.6% (56/149)
Morpheus v2	**0.094**	[0.058, 0.131]	**60.4% (90/149)**
Latxa-Qwen3.5-2B	0.237	[0.201, 0.275]	68.5% (102/149)

Key findings:

1. **Latxa has the highest CSR** (0.237) and word accuracy (68.5%), consistent with its superior BPC. Its CIs do not overlap with Morpheus or GPT-2.
2. **GPT-2 and Morpheus have overlapping CIs** — the CSR difference (0.110 vs 0.094) is not statistically significant. However, **Morpheus has 1.6x higher word accuracy** (60.4% vs 37.6%), meaning it predicts the correct word far more often. This is another instance of the CSR paradox (Section 6.14): Morpheus predicts correct words more frequently but saves fewer keystrokes per correct prediction, because agglutinative Basque words are longer and require more characters before the model converges.
3. **Latxa produces noisy completions.** As an instruct model, Latxa generates web navigation artifacts, markdown headers, and repeated text when used for raw text completion (e.g., the prompt "Euskal Herriko" produces "bidaia-gida/Ibilbideak/Arriurdin mendia"). This highlights a key architectural advantage of Morpheus: as a **base language model** (not instruction-tuned), it is naturally suited for the autocomplete task, where the model must continue text seamlessly rather than follow instructions.
4. **Inference engineering adds 3.9x CSR.** Morpheus's simplified CSR of 0.094 increases to 0.362 with the full inference pipeline (retokenization fallback, sticky merge, top-k alternatives — see Section 5.5). This demonstrates that the engineering strategies documented in this paper are not marginal optimizations but a major contribution, nearly quadrupling the raw model's autocomplete utility.

Deployment Efficiency and Production Context Placing Morpheus in the context of production autocomplete systems reveals where it sits in the design space:

System	Params	Size	Training Data	Deployment	Architecture	Paradigm
Gboard (on-device NWP)	1.4M	1.4 MB	Federated (per-user)	On-device (mobile)	LSTM	Next-word
Gmail Smart Compose	~80M	Server-side	~8B messages (~320B+ tokens est.)	Cloud TPU (data center)	LSTM	Multi-token
GitHub Copilot	Multi-billion	Server-side	Proprietary code corpus	Cloud GPU (Azure)	Transformer (FIM-tuned)	Multi-token
GPT-2 eus-euscrawl	124M	~50 MB (Q4)	~423M tokens	On-device (desktop)	Transformer	Multi-token

System	Params	Size	Training Data	Deployment	Architecture	Paradigm
Morpheus v2	**91M**	**55 MB** (Q4_K_M)	**~10B tokens**	**On-device (desktop/laptop)**	**Mamba-2**	**Both**
Latxa-Qwen3.5-2B	1,882M	~1.2 GB (Q4)	Proprietary	High-end only	Qwen3.5 (instruct)	Multi-token

Morpheus occupies a novel position: comparable in parameter count to Smart Compose's ~80M LSTM, but running **on-device** rather than on Cloud TPUs — and targeting a morphologically complex language that neither Smart Compose nor Gboard handles. GitHub Copilot, the other major production multi-token continuation system, is entirely cloud-dependent: its multi-billion-parameter model requires an elaborate global proxy infrastructure (HTTP/2 multiplexing, request cancellation, streaming, geographic routing) to achieve <200ms latency (Cheney, 2025). Morpheus eliminates this entire class of infrastructure problems by running on-device. For the **desktop multi-token continuation** use case (text editors, email clients), the 55 MB footprint is well within consumer hardware constraints. For the **mobile next-word prediction** use case, Gboard's 1.4 MB model defines the target; our 91M (55 MB) model is too large for that context and would require distillation to ~5-10M parameters to be viable on mobile. The inference engineering strategies documented in Section 5.5 are designed to carry over to such a distilled model.

Morpheus achieves BPC within 0.148 bits/char of a 20x larger model (Latxa) while being deployable on consumer laptops at 55 MB. Latxa's 1.2 GB quantized footprint restricts it to high-end hardware, and its instruct-tuning produces noisy completions for the autocomplete use case.

6.16 Data Scaling Analysis: Training Data Requirements for Low-Resource Language Models

A natural question for any language model training effort is whether the training data budget was appropriately sized. We analyze this through the lens of established scaling laws and our empirical training trajectory.

Tokens-to-parameters ratio. Our model has 91M parameters (including embeddings) and was trained on ~10B tokens, yielding a ratio of approximately **110:1** (tokens per parameter). The relevant scaling law literature provides context:

Framework	Ratio	Source
Chinchilla (compute-optimal)	20:1	Hoffmann et al. (2022)
DeepSeek (quality-adjusted)	30:1	Bi et al. (2024)
Mosaic (inference-optimal)	190:1	Sardana et al. (2024)
MiniCPM (small-model optimal)	192:1	Hu et al. (2024)
Morpheus (actual)	**110:1**	—

Chinchilla's 20:1 ratio is **compute-optimal** — it minimizes training compute for a target loss, but does not account for inference cost. Sardana et al. (2024) showed that for models with significant inference demand (~1B requests), the optimal ratio shifts upward: train smaller models on more

data than Chinchilla recommends. Since our model is deployed as a per-keystroke autocomplete system (exactly the inference-heavy scenario Mosaic addresses), the Mosaic/M iniCPM range is more applicable than Chinchilla. Hu et al. (2024) specifically studied small language models (0.04B–2B parameters) and found a much higher optimal ratio (192:1) than Chinchilla, contradicting the intuition that small models need less data.

By modern small-model standards, our 110:1 ratio is not excessive. It sits below both the Mosaic inference-optimal (190:1) and MiniCPM small-model-optimal (192:1) recommendations. Modern small language models are trained at far higher ratios: Qwen3-0.6B at 60,000:1 (36T tokens), Liquid LFM2.5-350M at 80,000:1 (28T tokens).

Empirical convergence analysis. While the ratio is reasonable by scaling-law standards, our training trajectory reveals where the model actually stopped learning:

Training segment	Tokens seen	Delta PPL	Tokens per 1.0 PPL improvement
Step 32K -> 54K	4.2B -> 7.1B	-0.39	7.4B
Step 54K -> 74K	7.1B -> 9.7B	-0.04	137.6B

The improvement rate dropped by **18.6x** between segments. The model effectively converged around **step 67K (~8.8B tokens)** — the final ~1.2B tokens produced no measurable PPL improvement. This does not mean the data budget was excessive; it means the **model capacity saturated on this data distribution**.

Data quality as the binding constraint. The convergence at 8.8B tokens is better explained by data quality than by data quantity. Three pieces of evidence converge:

1. **Corpus noise.** Our corpus quality audit (Section 4.2, Section 6.12) identified ~20–30% visible artifacts: emoji-heavy social media residue (18% of sampled lines), exact duplicates (5.2%), mixed-language content (6.4%), templated repetition, and date/number patterns. If ~20–30% of the corpus is low-value noise, the effective high-quality data is ~7–8B tokens — closely matching the 8.8B convergence point.

2. **GPT-2 eus-euscrawl as a natural experiment.** GPT-2 eus-euscrawl was trained on only ~423M tokens (3.4:1 ratio, heavily undertrained by Chinchilla standards) yet achieves nearly the same BPC as Morpheus (0.981 vs 0.970). However, Morpheus achieves **1.6x higher word accuracy** (60.4% vs 37.6%) — the 24x data advantage produces a dramatically better autocomplete model. This confirms that token count matters, but the BPC near-tie with a model trained on 24x less data suggests that character-level metrics saturate well before the model has learned the morphological patterns needed for good autocomplete. Quality multiplies the effectiveness of quantity.

3. **DeepSeek's quality finding.** Bi et al. (2024) found that "the data quality significantly influences the optimal model/data scaling up allocation strategy" — higher-quality data means the compute budget should shift toward model size rather than data size, implying that high-quality data is more efficient per token.

Implication: data quality over quantity. A 2–5B token corpus of aggressively filtered, high-quality Basque text would likely match or exceed the current 10B token mixed-quality corpus, because the marginal ~1.2B tokens that produced no improvement and the ~2–3B tokens of noise

were not contributing to model quality. The Chinchilla-optimal point for our model (~1.8B tokens) serves as a floor; the MiniCPM-optimal point (~17.5B tokens) is unattainable without more high-quality Basque data than currently exists. The practical sweet spot for low-resource agglutinative language modeling at this scale is **2–5B tokens of quality-filtered data**.

Smart Compose as a high-resource reference point. Google's Smart Compose (Chen et al., 2019) provides a concrete reference point for data scale at a similar parameter count. The production LSTM-2-1024 model (~80M parameters, comparable to our 91M) was trained on approximately **8 billion English email messages** with a word-level vocabulary of 50K English words (80/20 train/test split). While the paper reports message count rather than token count, even a conservative estimate of ~50 words per email yields ~400 billion word tokens (~320B training) — roughly **30–60x our 10B token corpus**. This enormous data advantage is available because English is a high-resource language and Google had access to the Gmail corpus of 1.5 billion users. For Basque, no comparable data volume exists: the total extant Basque text across all sources (Wikipedia, news, literature, web crawls, government gazettes) is finite and modest. This is the fundamental asymmetry between high-resource and low-resource language modeling — Google can scale data indefinitely; we cannot.

Beyond sheer volume, Smart Compose's training data has a second advantage that matters directly for user experience: **domain match**. The model is trained on emails and used to write emails — the training distribution and the deployment distribution are the same. Users typing in Gmail see suggestions calibrated to email phrasing ("I hope you're well", "let me know if you have any questions", "attached is the document"), because the model learned these patterns from billions of real emails. Morpheus, by contrast, is trained on **general Basque prose** (Wikipedia, news, literature, gazettes) but deployed as a general-purpose text editor autocomplete. This domain mismatch is the root cause of the corpus-induced artifacts documented in Section 6.12: the model over-predicts dates, numbers, and encyclopedic continuations because its training distribution is dominated by journalistic and encyclopedic text rather than the conversational or instructional prose that users typically type. Smart Compose's domain-matched data is itself a form of data quality — not just volume, but relevance to the deployment context — and one that is particularly difficult to obtain for Basque, where no large email or conversational corpus exists.

Our data scaling analysis thus operates in a qualitatively different regime: the question is not "how much data should we use?" but "given that Basque data is finite and domain-matched corpora do not exist, how do we maximize quality from what exists?" The answer, as argued above, is aggressive quality filtering rather than raw quantity maximization.

This finding has implications for low-resource language modeling more broadly: the field's emphasis on maximizing token count (web crawls, synthetic data, multilingual corpora) may be less productive than aggressive quality filtering of a smaller corpus. For languages like Basque where the total available high-quality text is finite and modest, data quality is the binding constraint, not data quantity. The Smart Compose comparison makes this concrete: a high-resource language model at the same parameter scale trains on 30–60x more data, but that data simply does not exist for Basque.

6.8 The Reconciliation: PPL Improves, Multi-Token Metrics Lack Power

Signal	Paradigm	Favors 74K?	Significant?	What it measures
PPL (1.83M held-out tokens)	—	[OK] Yes (7.56->7.17->7.13)	[OK] Yes (all 14 files agree)	Language model quality (full distribution)
CSR (300 held-out sentences)	Multi-token	Directionally (24.90->25.23->25.26)	[X] CIs overlap	Keystroke economy (lower bound)
Human A/B (30 blinded)	Multi-token	[X] No (11 vs 9 for 32K)	[X] p=0.82	Autocomplete quality (ground truth)
Completion replay (real usage)	Next-word	[OK] Yes (Top-1 60->80%)	—	Chip acceptance hit rate
Typing simulation (15 sentences)	Next-word	—	—	Word accuracy, CSR paradox

The core finding: PPL is the only metric that produces coherent, trustworthy signal in our experiments. It unambiguously confirms the final model (step 74K) is the best language model (7.56 -> 7.17 -> 7.13, all 14 files agree monotonically). The autocomplete-specific metrics proved unreliable: CSR requires meticulous implementation (token-ID prompts, no BOS, reference sentencepiece — all pitfalls we hit during development) and even when correctly implemented produces overlapping confidence intervals that cannot distinguish competent models; the human A/B is sample-limited (n=30, p=0.82) and underpowered to detect the improvement that PPL shows. The next-word replay does favor 54K (Top-1 60->80%), but it is built from a small number of sessions and conflates model quality with inference engineering. **In practice, PPL is the metric we trust for checkpoint comparison; the autocomplete metrics serve as sanity checks rather than ranking tools at this scale.** The CSR inversion documented in Section 6.15 — where NW-CSR actually *decreases* as PPL improves across the full training trajectory — provides the strongest evidence that autocomplete metrics can actively mislead checkpoint selection.

Why PPL and the autocomplete metrics diverge in reliability: - PPL measures the full next-token distribution across 1.83M tokens (smooth, low-variance, high statistical power, straightforward to implement correctly) - CSR and A/B are narrow probes: greedy single-path completion on 300 or 30 samples respectively (high-variance, low power, and easy to implement incorrectly — we discovered that string-prompt CSR gave ~4% vs ~28% with token-ID prompts due to BOS and tokenizer divergence bugs) - The exact-match/gold problem caps measurable CSR regardless of model quality — valid alternative Basque continuations are not credited - Many prompts have multiple valid continuations (the expert confirmed: ~33% ties), adding noise that obscures small improvements - Human A/B at n=30 cannot distinguish models: a 60/40 split yields p=0.30; even a 70/30 split yields p=0.13 - In summary: PPL is robust and easy to get right; the autocomplete metrics are fragile, sample-limited, and difficult to implement correctly — and even when correctly implemented, they lack the power to rank competent models

6.9 CSR is a Lower Bound for Agglutinative Autocomplete

CSR uses exact-match against a single gold string and cannot credit valid Basque alternative continuations. Once both models are "competent," CSR loses the resolution to distinguish them: it detects *regression* (garbage -> CSR crashes) but not *progress* between competent models at small sample sizes.

This was confirmed empirically: an earlier 30-sentence eval showed 54K as *worse* than 32K (29.17% vs 26.82%). At n=300 on genuinely held-out text, the direction flipped to agree with PPL (25.23% vs 24.90%). The 30-sentence result was pure noise from a saturated, too-small sample — exactly the failure mode predicted by the reform.

The typing simulation (Section 6.14) reveals a deeper problem: CSR is not merely resolution-limited, it is **structurally biased against the target language**. Agglutinative morphology requires longer typed prefixes before prediction, consuming the keystroke savings that CSR measures. The model's native Basque scores lower simulated CSR than two languages representing <1% of training data — a paradox that confirms CSR should not be used as a primary optimization target for agglutinative autocomplete.

6.10 Expectation Bias in Unblinded Assessment

The expert's earlier **unblinded** impression that 54K was "much better" was NOT borne out by **blinded** A/B evaluation (statistical tie). This is a classic expectation/anchoring bias.

> Unblinded qualitative assessment can mislead. The expert's unblinded impression that 54K was "much better" was not borne out by blinded A/B evaluation, which showed the two checkpoints as statistically equivalent at n=30. PPL confirmed 54K improved as a language model, and all directional evidence favors 54K, but the multi-token metrics lacked the power to confirm this at current sample sizes. Blinded evaluation is essential for honest model comparison.

6.11 Overall Assessment at Step 74K (Training Complete)

Metric	Result	Interpretation
Held-out PPL	7.13	Strong LM quality; converged (flat from step 67K)
CSR (macro, n=300)	25.26% [24.00, 26.52]	Meaningful keystroke savings; saturated (barely moved from 54K)
MorphAcc	76%	Strong morphological competence; saturated at 54K (tokenizer-bound)
Paradigm Hit@5	27.4%	Partial case system; noisy across checkpoints
Human A/B	Tie (p=0.82)	Statistical tie at n=30 (underpowered to distinguish)
Q4_K_M size	55 MB	Well within 300 MB budget

Key insight: The model provides meaningful keystroke savings and strong morphological competence. PPL improved monotonically throughout training (7.56 -> 7.17 -> 7.13) and is the only metric that unambiguously tracks model quality. The autocomplete metrics (CSR, MorphAcc, Paradigm, A/B) all saturated or are noisy — they serve as sanity checks but cannot reliably rank

the checkpoints. Crucially, the 54K->74K improvement is visible only in PPL (7.17 -> 7.13): CSR barely moves (25.23% -> 25.26%), MorphAcc is flat (76% -> 76%), and Paradigm is noisy (Hit@1 actually drops). This confirms that once a model reaches competence, only PPL has the resolution to measure further improvement. The model is fully converged; no metric shows room for further gains without architectural or data changes.

6.12 Corpus-Induced Prediction Artifacts

A persistent quality issue observed during qualitative testing is the model's tendency to autocomplete with **dates, numbers, and temporal expressions** in contexts where a human would predict more general continuations. For example:

Prompt	Model suggestion	Expected style
`Aipatu bezala,`	`2015eko ekainean,`	General continuation

The model predicts `2015eko ekainean` (*"in June 2015"*) rather than a more broadly useful continuation. This is not a random error but a **systematic corpus bias**: the training corpus is dominated by encyclopedic (Wikipedia) and journalistic (Berria) prose, where sentences following phrases like *"as mentioned," "as stated,"* or *"according to"* overwhelmingly reference specific dates, years, and quantities. The model has faithfully learned this distribution.

This artifact persists despite the exclusion of official gazette sources (BOG, BOPV, BOTHA), which were removed precisely because they contained bare numbers in sentence position (decree IDs, budget amounts). The residual date/number bias comes from **legitimate prose** — Wikipedia articles and news articles are inherently rich in temporal and numeric references. This represents a fundamental tension: the same encyclopedic and journalistic sources that provide high-quality, well-formed Basque prose also teach the model that dates and numbers are highly predictable continuations. It is also a manifestation of **domain mismatch**: the model is trained on encyclopedic and journalistic prose but deployed as a general-purpose text editor autocomplete. Google's Smart Compose avoids this problem entirely because its training data (emails) matches its deployment context (writing emails) — the model learns exactly the distribution users produce (Section 6.16). For Basque, no large conversational or email corpus exists, so we train on the best available general prose and accept the resulting domain artifacts as a known limitation.

Implications for evaluation: This artifact is not captured by PPL (predicting frequent date patterns *lowers* PPL) and is only partially captured by CSR (the held-out sentences may or may not contain dates). It is most visible in open-ended autocomplete testing with real user prompts, where the mismatch between the model's learned distribution and the user's intent becomes apparent. This reinforces the finding that **no single metric is sufficient** and that qualitative testing with domain experts remains essential — a conclusion independently reached by the GitHub Copilot team, who report that "language-specific evaluations lead to better outcomes along quality and style preferences" beyond what execution-based testing, LLM-based evaluations, and A/B testing provide (Fu & Mogensen, 2026).

Mitigation directions are discussed in Section 7.1 (data-level) and Section 7.3 (domain fine-tuning).

6.13 Cross-Lingual Transfer

Although the model is trained exclusively on Basque-focused corpora, the training data inevitably contains small amounts of non-Basque text. Web-crawled sources (CC-100-derived Colossal-OSCAR, Cultura-X, FineWeb-2, HPLT) and parliamentary transcripts (Parlaeus) include English and Spanish passages embedded within Basque documents — a reflection of the multilingual reality of the Basque Country. Corpus sampling estimates approximately **0.6% English content** by weighted volume; Spanish content is similarly present, particularly in parliamentary and web-crawl sources.

A natural question is whether this incidental multilingual exposure produces any cross-lingual capability. We tested next-word prediction on common collocations in English, Spanish, and Basque using the keyboard-mode demo endpoint (top-3 candidates, retokenization fallback enabled). Results:

Language	N prompts	Top-1	Top-3	Avg. max confidence
Basque	10	**60.0%**	**80.0%**	**0.511**
English	30	23.3%	23.3%	0.196
Spanish	30	16.7%	30.0%	0.299

The model is **strongly Basque-specialized**. On expert-authored Basque prompts testing auxiliary verb agreement and common collocations, the model achieves 60% top-1 and 80% top-3 accuracy with 51% average confidence — roughly 3x the top-1 accuracy and 1.7x the confidence of either English or Spanish. The Basque prompts include ergative alignment contrasts that the model resolves correctly:

Prompt	Expected	Predicted (top-1)	Correct?
`Azkenean guk ezin izango`	`dugu`	`dugu`	[OK]
`Azkenean gu ezin izango`	`gara`	`gara`	[OK]

The model correctly predicts `dugu` (1st-person plural transitive, ergative `guk`) after the transitive frame and `gara` (1st-person plural intransitive, absolutive `gu`) after the intransitive frame — a non-trivial morphological distinction at the core of Basque grammar.

Word-completion also functions correctly for Basque suffix attachment:

Prompt	Expected	Predicted (top-1)	Correct?
`Mila esker zure arreta`	`gatik -> arretagatik`	`arretagatik`	[OK]
`Aldez`	`aurretik`	`aurretik`	[OK]
`Edonola`	`ere`	`ere`	[OK]

Despite this Basque dominance, the model does exhibit **weak incidental cross-lingual transfer**. It correctly predicts several well-known English and Spanish collocations:

Language	Prompt	Expected	Predicted (top-1)	Correct?
English	`Thank you very`	`much`	`much`	[OK]
English	`The United States of`	`America`	`america`	[OK]
English	`Hello how are`	`you`	`you`	[OK]
Spanish	`Los Estados`	`Unidos`	`unidos`	[OK]
Spanish	`América del`	`Sur`	`sur`	[OK]
Spanish	`Es importante`	`que`	`que`	[OK]

These are high-frequency collocations that appear in multilingual web text. The model has learned the associations but with low confidence (English 0.196, Spanish 0.299 vs Basque 0.511) and fails on less formulaic phrases. This is consistent with the nature of web-crawled Basque corpora, where English and Spanish appear as embedded quotes, technical terms, and mixed-language passages rather than as coherent monolingual text.

This cross-lingual transfer is an **artifact of corpus composition, not a feature**. It does not diminish the model's Basque specialization — the 3x accuracy gap and 1.7x confidence gap confirm that the model's predictive capability is concentrated on Basque. However, it does highlight a property of low-resource language modeling: even aggressively monolingual corpora contain enough multilingual contamination to produce measurable cross-lingual effects, a consideration for future work on purer training data (Section 7.1).

The sentence-level typing simulation (Section 6.14) further reveals that this transfer is **functional, not merely collocational**: the model sustains full 10–12 word English and Spanish sentences with 100% word accuracy and high confidence on individual words (e.g., *language* at 0.993, *trabajando* at 0.991). This suggests that even incidental multilingual exposure produces usable next-word prediction for high-entropy collocations in non-target languages.

6.14 The CSR Paradox: Agglutinative Morphology Penalizes Native-Language Keystroke Savings

To evaluate the keyboard-mode autocomplete under realistic usage, we developed a typing simulation that **faithfully replicates the frontend algorithm** from the predictive keyboard demo (Section 5.5): char-by-char typing, sticky merge (candidate carry-forward with +0.1 prob boost), top-3 chip display from a top-5 fetch, and the full acceptance semantics (auto-space on word acceptance, punctuation attachment, next-word insertion with leading space). The simulation queries the same backend (`_keyboard_candidates`) as the live WebSocket endpoint. Fifteen sentences — five each in Basque, English, and Spanish — are translations of the same semantic content, controlling for topic and sentence structure across languages. The model accepts a suggestion the moment a matching candidate appears in the top-3 chips; otherwise it types the next character.

Results:

Language	Words correct	Simulated CSR	Avg. confidence	Acceptance rate	Avg. prefix before accept
Basque (native)	39/39 (100%)	**19.2%**	0.204	77%	**4.7 chars**

Language	Words correct	Simulated CSR	Avg. confidence	Acceptance rate	Avg. prefix before accept
English (<1% corpus)	50/50 (100%)	26.3%	0.463	68%	3.1 chars
Spanish (<1% corpus)	46/46 (100%)	**30.9%**	**0.590**	70%	3.1 chars
Overall	135/135 (100%)	25.4%	0.424	71%	3.6 chars

The model's native language achieves the lowest simulated CSR — below two languages that represent less than 1% of the training corpus. Spanish, the least-represented language, achieves the highest CSR. This is counterintuitive: one would expect the model to perform best on the language it was trained on.

This is **not a model deficiency**. It is a structural property of agglutinative morphology, and it reveals a fundamental flaw in using CSR as a primary metric for agglutinative autocomplete.

Root cause: morphological length. Basque words are longer and morphologically complex. The model needs an average of 4.7 typed characters before the correct Basque suggestion appears, versus 3.1 for English and Spanish. A word like *paseatzera* ("to go for a walk") cannot be predicted from *pa* — the model must see *paseatzer* before it confidently offers the full word. In contrast, English *walk* is predictable from *w* once the collocational context (*go for a...*) is established. The keystroke savings are consumed by the longer prefix the user must type before a suggestion becomes available, even though the model ultimately predicts the correct word.

Confidence inversely correlates with CSR. Basque has the lowest average confidence on accepted suggestions (0.204) yet the highest acceptance rate (77%). The model is "cautiously correct" on Basque — it identifies the right word but with distributed probability mass across morphological variants (e.g., *paseatzera*, *paseatzeko*, *paseatzen* are all valid continuations of *paseatze*). English and Spanish, with shorter words and stronger collocational patterns, produce high-confidence predictions (English *language* at 0.993, Spanish *trabajando* at 0.991) but lower acceptance rates — many short function words (*the, is, a / el, y, un*) are too ambiguous at 1–2 characters to predict, dragging down the acceptance rate without reflecting on model quality.

Only **one high-confidence (>=0.8) acceptance occurred in Basque** (*zait*, 0.932), compared to 11 in English and 11 in Spanish. Yet Basque achieved 100% word accuracy — the model was always correct, just less certain. This is the signature of agglutinative prediction: multiple valid morphological continuations distribute probability mass, producing lower per-word confidence without indicating poorer predictions.

Implications for evaluation methodology. This finding demonstrates that CSR is **not merely a lower bound** for agglutinative autocomplete (Section 6.9) — it is a **structurally biased metric** that penalizes the very language the system is designed to serve. If CSR were used as a primary optimization target, it would systematically favor shorter-word languages and mislead development away from the target language. The paradox arises because CSR measures keystroke economy, which is a function of both model quality and language typology: agglutinative languages require longer prefixes before prediction is possible, and no amount of model improvement can fully overcome this

— a 15-character Basque word will always require more typed characters before confident prediction than a 4-character English word. This aligns with GitHub's production experience with Copilot code completion (Fu & Mogensen, 2026), where they found that optimizing for acceptance rate alone "could lead to incorrectly favoring a high volume of simple and short suggestions" — the same structural bias we identify here, observed independently at production scale in a different domain (code vs. natural language).

This has a practical consequence for the community: **reporting CSR for agglutinative autocomplete without the context of morphological word length is misleading.** A naive reader comparing our 25.4% simulated CSR to the ~80% achievable in English autocomplete would conclude the model is poor, when in fact it achieves 100% word accuracy and 77% acceptance on its target language. The gap is structural, not qualitative. We recommend that CSR for agglutinative languages be accompanied by (1) word accuracy (does the model ever suggest wrong words?), (2) acceptance rate (how often does a suggestion match the user's intent?), and (3) average prefix length before acceptance (how much must the user type before prediction engages?). These metrics together provide a fairer picture than CSR alone.

6.15 The CSR Inversion: Autocomplete Metrics Move Opposite to Model Quality

The CSR paradox (Section 6.14) shows that CSR penalizes agglutinative languages structurally. A more troubling question is whether CSR tracks model quality *within* a single language across training. To test this, we ran the next-word CSR simulation (the PyTorch port of the keyboard algorithm, Section 5.5.6) on **seven checkpoints** spanning the full training trajectory, from step 10K (early training) through step 76K (converged). The same 30 Basque CSR test sentences were used at every checkpoint, and the simulation faithfully replicates the deployed keyboard algorithm (retokenization fallback, sticky merge, top-3 display, acceptance semantics).

Results across the full training trajectory:

Step	Held-out PPL	NW-CSR	95% CI	Word Acc	Acceptance	Avg Prefix	Confidence	Manual
10K	—	0.375	[0.313, 0.438]	1.000	0.859	3.5	0.408	14.1%
20K	—	**0.402**	[0.344, 0.469]	1.000	0.852	3.2	0.415	14.8%
30K	—	0.382	[0.330, 0.443]	1.000	0.839	3.4	0.422	16.1%
32K	7.56	0.397	[0.352, 0.450]	1.000	0.866	3.6	0.400	13.4%
54K	7.17	0.385	[0.355, 0.448]	1.000	0.859	3.6	0.415	14.1%
74K	**7.13**	**0.362**	[0.313, 0.426]	1.000	0.832	3.5	**0.433**	16.8%
76K	7.13	0.373	[0.320, 0.450]	1.000	0.832	3.3	0.432	16.8%

What is clear: the model improved throughout training. Held-out perplexity decreased monotonically from 7.56 (step 32K) to 7.13 (step 74K), a 5.7% reduction in cross-entropy loss. The training

trajectory is unambiguously positive — the final model is the best language model by a wide margin over the early checkpoints.

What is unexpected: NW-CSR does not track this improvement. It is non-monotonic across the full trajectory, peaking at step 20K (0.402) and reaching its lowest point at step 74K (0.362) — the best checkpoint by PPL. If CSR had been used as the checkpoint selection criterion, it would have selected step 20K over step 74K, choosing a dramatically worse language model. We call this the **CSR inversion**: the autocomplete metric moves in the *opposite direction* to model quality.

The decomposed metrics reveal a consistent pattern. **Confidence increases monotonically** (0.408 -> 0.433) while **acceptance rate decreases** (0.859 -> 0.832). The better model is *more confident* in its predictions, but those predictions match the gold target *less often*. Meanwhile, **word accuracy remains perfect** (1.000) at every checkpoint — the model never suggests an incorrect word, not even at step 10K. The decline in CSR is not quality degradation; it is something else entirely.

Hypotheses (preliminary, not yet confirmed). We do not yet understand why this inversion occurs, and we cannot rule out the possibility that our CSR computation itself contributes to the effect. We present several hypotheses without confidence:

1. **Multiple valid continuations.** As the model improves, it learns that Basque sentences have multiple valid continuations — different word orders, synonymous expressions, morphological variants. The model becomes confident in one valid continuation, but if it differs from the gold string, exact-match CSR cannot credit it. The increasing confidence (0.408 -> 0.433) coupled with decreasing acceptance (0.859 -> 0.832) is consistent with this: the model converges on *a* valid answer, just not *the* gold answer. This would mean CSR is measuring convergence to a specific reference, not prediction quality.

2. **Agglutinative probability distribution.** In agglutinative languages, a partial word prefix (e.g., *paseatze*) has many valid morphological continuations (*paseatzera*, *paseatzeko*, *paseatzen*). A better model may distribute probability more accurately across these variants, producing lower top-1 confidence on any single one — yet the simulation accepts only on exact match. This would depress CSR for the better model without indicating worse predictions. The CSR paradox (Section 6.14) already demonstrated this effect cross-lingually; the inversion may be its within-language analogue.

3. **Metric computation artifacts.** We cannot yet rule out that the next-word CSR simulation itself introduces bias. The retokenization fallback queries shorter prefixes and merges candidates across paths; the probabilities from different fallback paths are not strictly comparable. Sticky merge carries forward candidates with a +0.1 boost, which may help or hurt depending on checkpoint-specific candidate rankings. The interaction between these heuristics and model quality is not well understood. A simpler metric — pure top-1 next-token accuracy without the full keyboard pipeline — would help isolate whether the inversion is in the model or in the simulation.

4. **Evaluation set sensitivity.** The 30 CSR test sentences are drawn from Wikipedia text about a single institution (Euskal Herriko Unibertsitatea). As the model trains on more Wikipedia data, it may shift toward different stylistic continuations than those in the gold set — not because it is worse, but because it has learned the broader distribution. A larger, more diverse evaluation set would test this.

5. **Small sample size.** All confidence intervals overlap substantially (n=30 sentences, 149

words). The differences between checkpoints are not statistically significant. The inversion is a directional trend across 7 checkpoints, not a proven effect. It is possible that with a larger sample, the trend would flatten or reverse.

What we can conclude with confidence. Regardless of the explanation, the practical implication is clear: **CSR should not be used as a primary checkpoint selection criterion.** A metric that peaks at step 20K and declines toward the converged model is not measuring what we want to optimize. Perplexity, which directly measures the model's probability assignment to held-out text, remains the only reliable metric for ranking checkpoints. The CSR inversion provides the strongest evidence yet for this claim: it is not merely that CSR *lacks power* to distinguish competent checkpoints (Section 6.8) — it can actively *prefer* a worse model.

GGUF cross-validation: the inversion is backend-dependent. To test whether the inversion is a property of the model or of the computation backend, we ran the same 30 CSR test sentences through the **deployed GGUF model** (Q5_K_M quantization, llama.cpp inference on GPU) using the same keyboard simulation but querying the demo server's `/api/autocomplete/keyboard` endpoint instead of the PyTorch model directly. The results are striking:

Step	Backend	NW-CSR	Acceptance	Confidence	Manual
32K	PyTorch (bf16)	**0.397**	0.866	0.400	13.4%
32K	GGUF (Q5_K_M)	**0.362**	0.852	0.372	14.8%
54K	PyTorch (bf16)	0.385	0.859	0.415	14.1%
54K	GGUF (Q5_K_M)	0.390	0.839	0.413	16.1%
74K	PyTorch (bf16)	**0.362**	0.832	**0.433**	16.8%
74K	GGUF (Q5_K_M)	**0.389**	0.852	0.381	14.8%

The CSR inversion **does not replicate in GGUF**. In PyTorch, CSR decreases monotonically with training (0.397 -> 0.385 -> 0.362); in GGUF, CSR is roughly flat (0.362 -> 0.390 -> 0.389), and step 32K — which had the *highest* CSR in PyTorch — has the *lowest* in GGUF. The directional trend reverses between backends.

This finding directly supports hypothesis (3) above: **the inversion is partly a computation artifact, not a fundamental model property.** The PyTorch forward pass (bf16 autocast) and the llama.cpp inference path (Q5_K_M dequantization) produce different logprob distributions, which in turn produce different candidate rankings. The sticky merge and retokenization fallback amplify these differences because they query multiple prefixes and merge candidates whose probabilities are not strictly comparable across computation paths.

However, **both backends agree on the fundamental conclusion**: CSR does not track PPL improvement in either direction. Neither PyTorch nor GGUF shows CSR increasing with model quality. The claim that CSR should not be used as a primary checkpoint selection criterion holds regardless of backend — but the stronger claim that CSR *inverts* (actively prefers worse models) is backend-specific and should be stated with that caveat. Quantization also reduces confidence across all checkpoints (e.g., step 74K: 0.433 PyTorch -> 0.381 GGUF), as expected from information loss in 5-bit weight encoding.

Word accuracy remains perfect (1.000) in both backends at all checkpoints — the model never suggests an incorrect word regardless of quantization or inference engine. This reinforces that

the CSR variation is not about prediction quality but about the interaction between exact-match scoring, probability distributions, and computation backend.

---

7. Future Work

7.1 Immediate

1. **Training is complete** (76K steps, ~10B tokens, 14.9 hours). PPL decreased monotonically from 7.56 (step 32K) to 7.13 (step 74K, best checkpoint). The model is fully converged — validation loss is flat from step 67K onward (Delta < 0.001). The full evaluation suite has been run on the final checkpoint (Section 6.2–6.5): PPL, CSR (n=300), MorphAcc, and Case Paradigm all confirm that autocomplete metrics saturate while PPL continues to improve. The CSR inversion documented in Section 6.15 confirms that PPL remains the only reliable checkpoint-ranking metric. Next: investigate the CSR inversion with a larger, more diverse evaluation set (100+ sentences).
2. **Investigate repetition loops** — both checkpoints exhibit greedy-decoding repetition on some prompts. Repetition penalty or nucleus sampling may improve practical autocomplete quality more than further training.
3. **Mitigate date/number prediction artifacts** (Section 6.12). The model over-predicts temporal and numeric expressions because the corpus is dominated by encyclopedic and journalistic prose. Candidate mitigations include: (a) **downweighting lines with high digit density** during training, (b) **post-hoc filtering** of numeric-heavy suggestions in the demo server, or (c) **domain reweighting** to reduce the relative proportion of Wikipedia/news text in favor of more conversational or instructional prose. Each approach has tradeoffs: downweighting may harm legitimate date prediction, post-hoc filtering adds latency, and domain reweighting requires additional clean data sources.
4. **Scale CSR evaluation** — 300 sentences with CIs is a significant improvement over 30, but 1000+ would further tighten confidence intervals.

7.2 Near-term (requires morphological analyzer)

1. **Integrate Apertium Basque as the morphological analyzer.** Apertium is the only practical Basque tool that emits explicit morpheme boundaries (`+` symbols) rather than only lemmas or UD features. Throughput is ~9.4x10^5 words/second, sufficient for a 22 GB corpus pass. The output must be **surface-preserving**: use segmented forms like `etxe#tik` rather than underlying morphology (`etxe+a+tik`) that doesn't match the visible text.

2. **Pre-segment corpus with surface-preserving morpheme boundaries** and retrain a morphology-aware tokenizer. Based on QuechuaTok results, this should increase MorphAcc from ~67% toward ~80%+.

3. **Distill a mobile-variant model** for the next-word prediction paradigm (Section 5.4.1). The current 91M model (55 MB) is sized for the desktop multi-token continuation use case (comparable to Smart Compose's ~80M). For mobile next-word prediction, Gboard's 1.4M/1.4MB model defines the deployment target. A distilled Morpheus-Mobile (~5-10M parameters, ~5-10 MB) using the 91M model as teacher would bring the inference engineering strategies (Section 5.5) — retokenization fallback, sticky merge, top-k alternatives — into a model small enough for smartphone keyboards. The Mamba-2 architecture's O(1) memory

(no KV cache) is a key advantage for this target, as it eliminates the memory management complexity that Transformer-based mobile LMs (e.g., HuoziIME) must handle.

7.3 Medium-term

1. **Train Morpheus-Base (207M)** on the 4K tokenizer and compare against Morpheus-Small. A larger model may produce autocomplete improvements that are statistically detectable at current sample sizes — the 91M model's gains between 32K and 54K were directionally positive but too small for the multi-token metrics to confirm.
2. **MWE token injection**: Extract the 1,000 most frequent Basque multi-word expressions and inject them as single tokens. This directly reduces autoregressive decoding steps by 40-60% for covered phrases.
3. **Extend cross-model comparison** (Section 6.7): The current BPC comparison includes GPT-2 (124M), Latxa-Qwen3.5-2B (1.88B), and Morpheus (91M). A larger Latxa variant (7B+) and a base (non-instruct) Latxa model would provide a cleaner baseline, as the current Latxa's instruct-tuning produces noisy completions for raw text completion. Additionally, applying the full inference engineering pipeline (Section 5.5) to the baseline models would isolate the contribution of inference engineering from model quality.
4. **Domain-specific fine-tuning.** The base model is trained on general Basque prose (Wikipedia, news, literature), which produces a general-purpose autocomplete but also corpus-induced artifacts such as the date/number prediction bias documented in Section 6.12. A lightweight fine-tuning stage on domain-specific corpora — e.g., legal Basque (terminology, statute language), medical, educational, or conversational/informal text — could improve suggestion relevance for specialized use cases while also mitigating the general-corpus artifacts. The current training infrastructure supports pretraining from scratch and checkpoint resume (continuing the same run), but **does not yet support fine-tuning-specific features**: separate fine-tuning datasets, layer freezing, differential learning rates, or parameter-efficient methods such as LoRA. The 91M parameter scale is small enough that full fine-tuning on a single GPU is feasible; the on-device deployment constraint means domain-adapted variants could be distributed as separate GGUF files and hot-swapped at runtime via the demo server's model reload endpoint (Section 5.4). This is also a candidate mitigation for the Section 6.12 date/number artifacts: a domain adapter trained on conversational or instructional prose would shift the model's distribution away from the encyclopedic patterns that produce numeric predictions.
5. **Data scaling experiment for Basque.** The analysis in Section 6.16 suggests that 2–5B tokens of aggressively filtered, high-quality Basque text would likely match or exceed the current 10B token mixed-quality corpus. To empirically determine the optimal data size, train four models with identical architecture on 1B, 2B, 5B, and 10B token subsets of a quality-filtered corpus, and compare held-out PPL at convergence. This would yield the first data scaling law curve for Basque — a contribution to low-resource language modeling methodology. The experiment would also test whether the convergence at ~8.8B tokens (Section 6.16) is a property of data quality (fixable with better filtering) or of model capacity (requiring a larger model to benefit from more data).

7.4 Long-term

1. **Hybrid Mamba-Attention architecture**: If pure Mamba-2 struggles with long-range morphological agreement, add 2-4 attention layers ("Jamba-style" hybrid).

2. **Distillation from Kimu 9B**: If a distilled 200M Transformer from the 9B Basque teacher becomes feasible, compare against the pure Mamba baseline.
3. **User study with CSR measurement**: Deploy to real Basque speakers and measure actual keystroke savings with A/B testing.

---

8. Code Availability

All code, model checkpoints, and evaluation artifacts are publicly available at `github.com/itzune/morpheus-mamba`. The repository contains:

- **Training pipeline**: Mamba-2 training with atomic checkpointing, gradient accumulation, and W&B integration
- **Data pipeline**: Corpus cleaning, SentencePiece Unigram tokenizer training, and pretokenization with validation leakage prevention
- **Export pipeline**: PyTorch -> HuggingFace -> GGUF conversion with BOS-token configuration
- **Evaluation suite**: PPL, CSR with bootstrap confidence intervals, MorphAcc, case paradigm completion, and blinded A/B evaluation
- **Demo server**: Docker-based FastAPI inference server supporting both multi-token ghost-text continuation (Smart Compose–style) and next-word prediction (smartphone keyboard–style chips), with completion logging and checkpoint replay
- **Tokenizer ablation data**: Full per-word tokenization results across 4K/8K/16K/32K vocabularies

The primary model checkpoint (step 74K, fully trained) and all quantized GGUF variants (Q4_K_M, Q5_K_M) are included. The model is also published on HuggingFace: `itzune/morpheus` (safetensors format) and `itzune/morpheus-gguf` (GGUF format).

---

9. Conclusion

Morpheus demonstrates that **State Space Models (Mamba-2) are a viable architecture for on-device predictive autocompletion in agglutinative languages**. The central research question — whether a Gmail Smart Compose–equivalent multi-token continuation system (Google's ~80M LSTM served on Cloud TPUs) can run **entirely on a consumer laptop** for text editors — is answered affirmatively: our 91M Mamba-2 model (55 MB quantized) provides multi-token ghost-text autocompletion on-device with zero network calls, comparable in parameter scale to Smart Compose but without the data-center dependency. The system also supports a second paradigm — next-word chip prediction for mobile keyboards (the Gboard paradigm, where Gboard's 1.4M/1.4MB model defines the deployment target) — though the current 91M model is too large for mobile and would require distillation to match Gboard's footprint. The inference engineering strategies documented in Section 5.5 are designed to carry over to such a distilled mobile variant. After full training (76K steps, ~10B tokens, 14.9 hours), the best checkpoint (step 74K) achieves:

- **25.3% character savings rate** (95% CI [24.0%, 26.5%]) on 300 held-out sentences
- **76% morpheme boundary accuracy** — a dramatic improvement over the old 32K-vocab model's 20%, validating the vocabulary-size ablation
- **Held-out PPL of 7.13** — converged (flat from step 67K onward)

- **55 MB on-disk size** (Q4_K_M) — deployable on consumer laptops for the desktop multi-token continuation use case (comparable to Smart Compose's ~80M, but on-device rather than server-side); **too large for mobile** next-word prediction, where Gboard's 1.4 MB model defines the target

Primary Contributions

1. **Vocabulary-size ablation for Basque.** We provide the first quantitative evidence that Unigram MorphAcc consistency drops from 66.7% at 4K to 28.6% at 32K on Basque, mirroring the QuechuaTok finding. At 4K, verbal agreement morphemes (`zki`, `zu`, `ke`) are independently accessible tokens; at 32K, they are buried inside opaque atomic units. **Fertility is a confound, not a quality metric, for agglutinative tokenizer evaluation.**
2. **Multi-metric evaluation suite for agglutinative autocomplete.** We combine PPL, CSR with bootstrap CIs, MorphAcc, Case Paradigm Completion, blinded human A/B, and completion logging with replay — each tagged to the prediction paradigm it measures. Each metric has known limitations; used in concert, they triangulate model quality. **In practice, PPL was the only metric that produced coherent, reliable signal for checkpoint ranking; the autocomplete metrics served as sanity checks rather than ranking tools at this scale.**
3. **CSR is a lower bound for agglutinative autocomplete — and structurally biased against the target language.** Exact-match gold cannot credit valid alternative Basque continuations. Once models reach competence, CSR loses resolution and cannot rank them at small sample sizes. This was confirmed empirically: a 30-sentence eval showed 54K as *worse* (noise); at n=300, the direction flipped to agree with PPL. **CSR detects regression but cannot measure progress between competent models without much larger sample sizes.** Furthermore, a sentence-level typing simulation reveals a **CSR paradox**: the model's native Basque achieves the lowest simulated CSR (19.2%), below English (26.3%) and Spanish (30.9%) despite those languages representing <1% of training data. This is a structural artifact of agglutinative word length — Basque words require 4.7 typed characters on average before correct prediction versus 3.1 for English/Spanish — not a model deficiency. **CSR penalizes the very language such systems are designed to serve, and should not be used as a primary optimization target for agglutinative autocomplete.**
4. **PPL is the only reliable metric; autocomplete metrics are fragile.** PPL improved monotonically from step 32K to the converged step 74K (7.56 -> 7.17 -> 7.13, all 14 files agree) and is the only metric that unambiguously ranked the checkpoints. The autocomplete metrics proved difficult to implement correctly (CSR gave ~4% with string prompts vs ~28% with token IDs due to BOS and tokenizer divergence bugs) and, even when correctly implemented, lacked the power to confirm the improvement: CSR confidence intervals overlap at n=300 (24.90% -> 25.23% -> 25.26%), MorphAcc saturates at 76% from step 54K onward, and the blinded A/B (p=0.82) is underpowered at n=30. The CSR inversion (Section 6.15) shows that next-word CSR can actively *decrease* as the model improves. **PPL is robust and easy to get right; the autocomplete metrics are fragile, sample-limited, and difficult to implement correctly — and even when correct, they lack the power to rank competent models. This is a practical lesson for the community: do not rely on CSR or small-n human A/B as primary checkpoint-ranking tools for agglutinative autocomplete.** This finding is corroborated by GitHub's production experience with Copilot,

where the team found that acceptance-rate optimization was structurally misleading and pivoted to "accepted and retained characters" as their primary metric (Fu & Mogensen, 2026) — the same lesson we reach here from a different starting point (agglutinative morphology rather than code completion scale).

5. **Expectation bias in unblinded assessment.** The expert's unblinded impression that 54K was "much better" was not borne out by blinded A/B evaluation. **Blinded evaluation is essential for honest model comparison.**

6. **Three-gate pre-training validation protocol.** A corpus audit, proxy overfit test, and autocomplete smoke test that jointly validate data quality, pipeline integrity, and inference-time behavior before committing GPU resources — complementing PPL with checks that target the failure modes PPL cannot detect.

7. **Inference engineering for agglutinative keyboards.** We document five strategies — retokenization fallback, sticky merge (candidate carry-forward), top-k exceeding display-k, next-word candidate extraction, and completion logging with replay — that address failure modes unique to deploying subword-tokenized models as interactive predictive keyboards. These strategies are validated on Basque but generalize to any agglutinative language where subword tokenization creates path-dependent prediction traps. We also report a critical inference engine dependency: Mamba-2 models require `llama.cpp` >= commit `dc2187d48` to avoid silently incorrect greedy outputs from an SSM scan bug.

8. **Cross-model baseline comparison using BPC.** We introduce Bits Per Character (BPC) as the correct tokenizer-independent metric for comparing language models with different vocabulary sizes. Morpheus (91M, BPC 0.970) outperforms GPT-2 eus-euscrawl (124M, BPC 0.981) despite fewer parameters — driven by 24x more training data — and approaches Latxa-Qwen3.5-2B (1.88B, BPC 0.822) at 1/20th the parameter count and 1/22nd the deployment size. Per-token PPL is misleading across vocabularies (GPT-2's 29.21 vs Morpheus's 9.83 suggests a massive gap that BPC reveals to be negligible). We also show that inference engineering adds 3.9x CSR on top of the raw model, demonstrating that the engineering strategies are a major contribution, not a marginal optimization.

9. **On-device Smart Compose feasibility for agglutinative languages.** We demonstrate that a Smart Compose–equivalent multi-token continuation system — which Google serves from Cloud TPUs using an ~80M LSTM trained on ~8 billion English email messages (~320B+ tokens estimated) — can run entirely on-device on a consumer laptop (91M Mamba-2, 55 MB, zero network calls, trained on ~10B tokens) for a morphologically complex language. The key architectural insight is shared with Google's production decision: both chose recurrent architectures (LSTM, Mamba-2) over Transformers to avoid KV-cache latency growth — but where Google solved the latency problem with data-center TPUs, Mamba-2's O(1) per-step inference solves it at the architecture level, enabling on-device deployment. We also position the two deployment regimes clearly: the 91M model is sized for desktop/text-editor use (comparable to Smart Compose), while mobile next-word prediction (Gboard: 1.4M/1.4MB) requires a distilled variant that has not yet been trained. The data scale asymmetry — Google's 30–60x larger training corpus — reflects the fundamental difference between high-resource and low-resource language modeling (Section 6.16).

10. **Data scaling analysis for low-resource language modeling.** We situate our 110:1 tokens-to-parameters ratio within the scaling law literature (Chinchilla 20:1 compute-optimal, Mosaic 190:1 inference-optimal, MiniCPM 192:1 small-model-optimal), finding that the ratio is

reasonable by modern small-model standards. However, empirical convergence analysis shows the model stopped improving at ~8.8B tokens — not because the data budget was excessive, but because **data quality is the binding constraint** in low-resource language modeling. A corpus audit revealing ~20–30% noise, combined with the GPT-2 eus-euscrawl comparison (24x less data, similar BPC, 1.6x lower word accuracy), suggests that 2–5B tokens of aggressively filtered high-quality Basque would match or exceed the current 10B mixed-quality corpus. **For low-resource languages where the total available high-quality text is finite, data quality — not data quantity — is the limiting factor.**

Limitations

- Training is complete (76K steps, ~10B tokens); the model is fully converged with PPL flat from step 67K onward
- CSR of 25% is below the ~80% achievable in English autocomplete, reflecting both the difficulty of agglutinative prediction, the exact-match metric's ceiling, and a **structural CSR paradox** (Section 6.14): the model's native Basque scores lower simulated CSR than non-target languages due to morphological word length, not model quality
- **Date/number prediction artifacts** (Section 6.12): the model over-predicts temporal and numeric expressions, a systematic bias inherited from the encyclopedic and journalistic training corpus. This degrades the practical utility of suggestions in contexts where users expect general continuations
- **Evaluation metric limitations (Section 6.8):** PPL is the only metric that produced coherent, reliable signal for checkpoint ranking. CSR is fragile to implement (BOS/tokenizer divergence bugs gave ~4% vs ~28%) and saturates at small sample sizes (25.23% -> 25.26% from 54K to 74K despite continued PPL improvement); MorphAcc saturates at 76% from 54K onward; human A/B at n=30 is underpowered (p=0.82). The autocomplete metrics served as sanity checks, not ranking tools. The CSR inversion (Section 6.15) shows that next-word CSR can actively move opposite to model quality. Larger-scale evaluation is needed for the autocomplete metrics to become statistically informative.
- No morphological pre-segmentation (Apertium) has been applied yet; MorphAcc could improve from 76% toward 83%+
- No user study; all evaluation is simulation-based or expert-judged
- The real-corpus PPL is contaminated (articles appear in training); absolute numbers are optimistic
- **The 91M model is too large for mobile deployment.** The desktop multi-token continuation use case (comparable to Smart Compose's ~80M) is well-served, but the mobile next-word prediction paradigm (Gboard: 1.4M/1.4MB) requires a distilled variant (~5-10M) that has not yet been trained
- **Data quality is the binding constraint, not data quantity** (Section 6.16). The model converged at ~8.8B tokens despite a 10B token budget, because ~20–30% of the corpus is noise (emoji, duplicates, mixed-language, templates). A 2–5B token high-quality subset would likely match or exceed current quality, but this has not been empirically tested

The Path Forward

The 4K tokenizer alone achieves 76% MorphAcc after full training — a 3.8x improvement over the 32K tokenizer's 20%, but still below the 83% that PRPE achieves on Quechua. The model has converged (PPL flat from step 67K), so further training on the same data distribution will

not close this gap. The data scaling analysis (Section 6.16) shows that the binding constraint is data quality, not data quantity: a smaller corpus of aggressively filtered, high-quality Basque text would likely be more efficient than the current 10B token mixed-quality corpus. The next step is to integrate **Apertium Basque** and pre-segment the corpus with **surface-preserving morpheme boundaries**, pushing MorphAcc toward 83%. For the mobile next-word prediction paradigm, a distilled Morpheus-Mobile (~5-10M) would bring the inference engineering strategies into a model matching Gboard's deployment footprint. The evaluation methodology presented here — combining PPL, CSR with CIs, MorphAcc, paradigm completion, blinded human A/B, and completion logging with replay — provides a replicable framework for measuring progress in agglutinative autocomplete. A key practical lesson is that **PPL was the only metric that produced coherent, reliable signal for checkpoint ranking at this scale; the autocomplete metrics (CSR, human A/B) proved fragile to implement correctly and underpowered even when correct**. This has implications for how the community evaluates models for morphologically rich languages: autocomplete-specific metrics require much larger sample sizes and careful implementation to become statistically informative, and PPL should not be dismissed merely because it does not measure end-to-end utility directly.

References

1. Contreras, M. (2026). *QuechuaTok: Morphological Boundary Accuracy as a Necessary Metric for Tokenizer Evaluation in Agglutinative Low-Resource Languages.* arXiv:2606.23943.
2. Arnett, C., Hudspeth, M., & O'Connor, B. (2025). *Evaluating Morphological Alignment of Tokenizers in 70 Languages.* arXiv.
3. Stephen, A., & Libovický, J. (2026). *Evaluating Morphological Plausibility of Subword Tokenization via Statistical Alignment with Morpho-Syntactic Features.* arXiv.
4. Xu, N., & Kim, A. (2026). *Tokenization and Morphological Fidelity in Uralic NLP: A Cross-Lingual Evaluation.* arXiv.
5. Táboas García, T., Przybyla, P., & Wanner, L. (2025). *Exploring morphology-aware tokenization: A case study on Spanish language modeling.* EMNLP 2025.
6. Hu, J. F. (2025). *Tokenization Strategies for Low-Resource Agglutinative Languages in Word2Vec: Case Study on Turkish and Finnish.* ACDSA 2025.
7. Etxaniz, J., Sainz, O., Perez, N., Aldabe, I., Rigau, G., Agirre, E., Ormazabal, A., Artetxe, M., & Soroa, A. (2024). *Latxa: An Open Language Model and Evaluation Suite for Basque.* arXiv:2403.20266.
8. Chen, M. X., et al. (2019). *Gmail Smart Compose: Real-Time Assisted Writing.* arXiv:1906.00080.
9. Hard, A., et al. (2018). *Federated Learning for Mobile Keyboard Prediction.* arXiv:1811.03604.
10. Fu, S., & Mogensen, J. (2026). *The Road to Better Completions: Building a Faster, Smarter GitHub Copilot with a New Custom Model.* GitHub Blog.
11. Cheney, D. (2025). *How GitHub Copilot Serves 400 Million Completion Requests a Day.* QCon San Francisco 2024 / InfoQ.
12. Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752.
13. Dao, T., & Gu, A. (2024). *Transformers are SSMs: Generalized Models and Efficient Algorithms through Structured State Space Duality.* arXiv:2405.21060.
14. Beck, M., et al. (2024). *xLSTM: Extended Long Short-Term Memory.* NeurIPS 2024.

15. Lane, W., Harrigan, A., & Arppe, A. (2022). *Interactive Word Completion for Plains Cree.* ACL 2022.
16. Trnka, K., & McCoy, K. (2008). *Evaluating Word Prediction: Framing Keystroke Savings.* ACL 2008.
17. Liquid AI (2026). *LFM-2.5-230M: On-Device State Space Models.* liquid.ai.
18. Orai NLP (2026). *Kimu: Basque-Adapted Language Models.* orai.eus.
19. ChaI-TeA (2024). *ChaI-TeA: A Benchmark for Chinese Input Method.* arXiv:2412.18377.
20. Kosyak, A., & Tyers, F. (2022). *Predictive text for agglutinative languages.*
21. WSTypist (2026). *Simulation-based mobile typing evaluation.* arXiv:2602.06489.
22. Rust, P., et al. (2021). *How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models.* ACL 2021.
23. Hoffmann, J., et al. (2022). *Training Compute-Optimal Large Language Models* (Chinchilla). arXiv:2203.15556.
24. Sardana, N., et al. (2024). *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws.* arXiv:2401.00448.
25. Hu, S., et al. (2024). *MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies.* arXiv:2404.06395.
26. Bi, X., et al. (2024). *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism.* arXiv:2401.02954.
27. Muennighoff, N., et al. (2023). *Scaling Data-Constrained Language Models.* arXiv:2305.16264.

This document is a living research record. Current as of July 11, 2026 — training complete (76,294 steps, best checkpoint step 74,000, PPL 7.13).